Serenity Styles

Beauty Salon Booking System

Mini Project Report

Submitted by

Deepthi C D

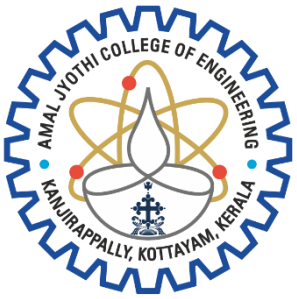
Reg. No.: AJC22MCA-I028

In Partial fulfillment for the Award of the Degree of

INTEGRATED MASTER OF COMPUTER APPLICATIONS

(INMCA)

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY



AMAL JYOTHI COLLEGE OF ENGINEERING

AUTONOMOUS

KANJIRAPPALLY

[Affiliated to APJ Abdul Kalam Technological University, Kerala. Approved by AICTE, Accredited by NAAC. Koovappally, Kanjirappally, Kottayam, Kerala 686518]

2024-2025

DEPARTMENT OF COMPUTER APPLICATIONS

AMAL JYOTHI COLLEGE OF ENGINEERING AUTONOMOUS

KANJIRAPPALLY

CERTIFICATE

This is to certify that the Project report, "**Serenity Styles"** is the bona fide work of **Deepthi C D (Regno: AJC22MCA-I028)** carried out in partial fulfillment of the requirements for the award of the **Degree of Integrated Master of Computer Applications** at **Amal Jyothi College of Engineering Autonomous, Kanjirappally,** Affiliated to **APJ Abdul Kalam Technological University**. The project was undertaken during the period from **January 01, 2025 to April 23, 2025.**

Ms. Nimmy Francis
Internal Guide

Ms. Meera Rose Mathew
Coordinator

Rev. Fr. Dr. Rubin Thottupurathu Jose
Head of the Department

DECLARATION

I hereby declare that the project report **"Serenity Styles"** is a bona fide work done at **Amal Jyothi College of Engineering Autonomous, Kanjirappally**, Affiliated to **APJ Abdul Kalam Technological University**, towards the partial fulfilment of the requirements for the award of the **Integrated Master of Computer Applications (INMCA)** during the period from **January 01, 2025 to April 23, 2025.**

Date: 23/04/2025 **Deepthi C D**

KANJIRAPPALLY **Reg: AJC22MCA-I028**

ACKNOWLEDGEMENT

First and foremost, I thank God almighty for his eternal love and protection throughout the project. I take this opportunity to express my gratitude to all who helped me in completing this project successfully. It has been said that gratitude is the memory of the heart. I wish to express my sincere gratitude to our Director (Administration) **Rev.Fr. Dr. Roy Abraham Pazhayaparampil** and Principal **Dr. Lillykutty Jacob** for providing good faculty for guidance.

I owe a great depth of gratitude towards our Head of the Department **Rev.Fr.Dr.Rubin Thottupurathu Jose** for helping us. I extend my whole hearted thanks to the project coordinator **Ms. Meera Rose Mathew** for her valuable suggestions and for overwhelming concern and guidance from the beginning to the end of the project. I would also express sincere gratitude to my guide **Ms. Nimmy Francis** for her inspiration and helping hand.

I thank our beloved teachers for their cooperation and suggestions that helped me throughout the project. I express my thanks to all my friends and classmates for their interest, dedication, and encouragement shown towards the project. I convey my hearty thanks to my family for the moral support, suggestions, and encouragement to make this venture a success.

Deepthi C D

ABSTRACT

The **Serenity Styles** is a web application that simplifies appointment scheduling at beauty salons. It allows users to create accounts, purchase products, browse services, and book appointments in real-time, ensuring availability through a dynamic interface. Customers can select services, choose time slots, and securely pay via integrated gateways like PayPal or Stripe. Salon administrators can manage appointments, services, and user accounts through an Admin Dashboard, while customer feedback is captured via ratings and reviews. The system features real-time availability checking, service filtering, and a responsive design for all devices. It enhances both the user and salon experience with efficient booking and management tools.

CONTENT

SL. NO	TOPIC	PAGE NO
1	INTRODUCTION	1
1.1	PROJECT OVERVIEW	2
1.2	PROJECT SPECIFICATION	2
2	SYSTEM STUDY	4
2.1	INTRODUCTION	5
2.2	EXISTING SYSTEM	5
2.3	DRAWBACKS OF EXISTING SYSTEM	6
2.4	PROPOSED SYSTEM	7
2.5	ADVANTAGES OF PROPOSED SYSTEM	8
3	REQUIREMENT ANALYSIS	9
3.1	FEASIBILITY STUDY	10
3.1.1	ECONOMICAL FEASIBILITY	10
3.1.2	TECHNICAL FEASIBILITY	10
3.1.3	BEHAVIORAL FEASIBILITY	11
3.1.4	FEASIBILITY STUDY QUESTIONNAIRE	11
3.2	SYSTEM SPECIFICATION	15
3.2.1	HARDWARE SPECIFICATION	15
3.2.2	SOFTWARE SPECIFICATION	15
3.3	SOFTWARE DESCRIPTION	16
3.3.1	PHP	16
3.3.2	MYSQL	16
4	SYSTEM DESIGN	17
4.1	INTRODUCTION	18

4.2	**UML DIAGRAM**	**18**
4.2.1	**USE CASE DIAGRAM**	**19**
4.2.2	**SEQUENCE DIAGRAM**	**20**
4.2.3	**STATE CHART DIAGRAM**	**22**
4.2.4	**ACTIVITY DIAGRAM**	**23**
4.2.5	**CLASS DIAGRAM**	**25**
4.2.6	**OBJECT DIAGRAM**	**26**
4.2.7	**COMPONENT DIAGRAM**	**27**
4.2.8	**DEPLOYMENT DIAGRAM**	**29**
4.3	**USER INTERFACE DESIGN USING FIGMA**	**30**
4.4	**DATABASE DESIGN**	**33**
5	**SYSTEM TESTING**	**46**
5.1	**INTRODUCTION**	**47**
5.2	**TEST PLAN**	**47**
5.2.1	**UNIT TESTING**	**48**
5.2.2	**INTEGRATION TESTING**	**48**
5.2.3	**VALIDATION TESTING**	**48**
5.2.4	**USER ACCEPTANCE TESTING**	**49**
5.2.5	**AUTOMATION TESTING**	**49**
5.2.6	**SELENIUM TESTING**	**49**
6	**IMPLEMENTATION**	**69**
6.1	**INTRODUCTION**	**70**
6.2	**IMPLEMENTATION PROCEDURE**	**70**
6.2.1	**USER TRAINING**	**70**
6.2.2	**TRAINING ON APPLICATION SOFTWARE**	**71**
6.2.3	**SYSTEM MAINTENANCE**	**71**

6.2.4	**HOSTING**	**71**
7	**CONCLUSION & FUTURE SCOPE**	**67**
7.1	**CONCLUSION**	**77**
7.2	**FUTURE SCOPE**	**77**
8	**BIBLIOGRAPHY**	**79**
9	**APPENDIX**	**81**
9.1	**SAMPLE CODE**	**82**
9.2	**SCREEN SHOTS**	**131**

List of Abbreviations

PHP	:	Hypertext Preprocessor
HTML	:	HyperText Markup Language
CSS	:	Cascading Style Sheets
JS	:	JavaScript
AJAX	:	Asynchronous JavaScript and XML
JQuery	:	JavaScript Query
DBMS	:	Database Management System
RDBMS	:	Relational DataBase Management System
SQL	:	Structured Query Language
UML	:	Unified Modeling Language
URL	:	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW

The Serenity Styles is a web-based appointment scheduling and e-commerce platform designed to enhance the booking experience at beauty salons. The system provides an intuitive interface for customers to register, explore services, book appointments in real-time, and purchase beauty products seamlessly. With integrated payment gateways like PayPal and Stripe, users can securely complete transactions while benefiting from a responsive and user-friendly design. Salon administrators can efficiently manage appointments, services, and customer accounts through a comprehensive Admin Dashboard. The system also includes tools for product management, financial tracking, and customer engagement through notifications, reminders, and feedback moderation.

A real-time availability checker ensures that customers can book only available slots, while service filtering enhances discoverability. The platform also supports ratings and reviews, allowing customers to share experiences and helping salons improve service quality. With its robust booking and management features, The Serenity Styles streamlines operations for salons, improves customer satisfaction, and optimizes business efficiency.

1.2 PROJECT SPECIFICATION

The System consist of two actors. They are:

Admin Dashboard Functionalities

1. Manage Appointments: View, approve, cancel, and reschedule appointments.
2. Service Management: Add/edit/delete services and categorize them.
3. User Accounts: Manage customer accounts, deactivate and activate users.
4. Product Management: Manage beauty products, prices, and stock.
5. Ratings & Reviews: View, moderate.
6. Financial Management: Track revenue and analyse sales.
7. Customer Support: Send emails /reminders.

User Functionalities

1. Registration/Login: Create an account or log in.
2. Browse Services: View and filter available services.

3. Book Appointments: Select services and book time slots.
4. Product Purchase: Browse and buy beauty products.
5. Ratings/Reviews: Rate and review services.
6. Order/Appointment History: View past bookings and purchases.
7. Account Management: Update personal details and preferences.
8. Payments: Pay securely for services and products.
9. Notifications: Receive reminders for appointments and offers.

CHAPTER 2

SYSTEM STUDY

2.1INTRODUCTION

In the modern digital era, businesses across various industries are leveraging technology to enhance customer experience and streamline operations. The **Serenity Styles** web application is designed to revolutionize the beauty salon industry by providing an efficient and user-friendly platform for appointment scheduling and product purchases. Traditional salon booking methods often involve phone calls or walk-ins, which can lead to inefficiencies such as scheduling conflicts, long wait times, and difficulty in managing customer preferences. This project aims to eliminate these challenges by introducing a real-time, automated booking system that ensures seamless scheduling and service availability.

Serenity Styles enables users to create accounts, browse salon services, book appointments, and purchase beauty products through an intuitive interface. Customers can select preferred services, choose suitable time slots, and make secure payments via integrated gateways like Razor pay. Additionally, users can provide feedback through ratings and reviews, enhancing service quality.

The primary goal of this project is to develop an efficient digital solution that enhances both the customer experience and salon management process.

2.2EXISTING SYSTEM

In our existing system, all the appointment bookings and customer details for the beauty salon are managed manually. As a result, it takes more time to schedule, update, or cancel an appointment. Customers are required to call or visit the salon in person to make a booking, which is not always convenient and often leads to miscommunication or missed appointments.

Another major disadvantage is that checking staff availability, tracking the services offered, and managing customer records is time-consuming and inefficient. Preparing reports for daily bookings or reviewing customer history involves going through physical registers or spreadsheets manually. After conducting a feasibility study, we decided to convert the manual salon management system into a computerized Beauty Salon Booking System to enhance efficiency, save time, and improve the overall customer experience.

2.2.1 NATURAL SYSTEM STUDIED

A natural system study for a beauty salon booking website involves analyzing how the platform interacts with its users and the broader salon industry. This includes understanding the key participants, such as customers, salon administrators, and payment service providers, and examining how they engage with the system. The study explores how users browse services, book appointments, purchase beauty products, and make secure payments, ensuring a seamless experience.

Additionally, it considers how the platform adapts to changing customer demands, seasonal trends, and new beauty service offerings. The interactions between salons, suppliers, and customers play a crucial role in ensuring smooth business operations. The study also examines the impact of the system on the beauty and wellness industry, including how it enhances customer convenience, streamlines salon management, and contributes to economic growth through job opportunities.

2.2.2 DESIGNED SYSTEM STUDIED

The focus is on structuring the functionalities to enhance user experience and streamline salon operations. The system architecture includes a frontend interface for seamless user interaction, a backend system for managing appointments, services, and user data, and integration with payment gateways like Razor pay for secure transactions.

The user interface is designed for intuitive navigation, featuring service filtering, real-time availability checks, and appointment scheduling. The system also includes product management for adding, updating, and managing beauty products available for purchase. User authentication ensures secure account registration, login, and profile management, enabling customers to track appointments, purchase history, and preferences.

2.3 DRAWBACKS OF EXISTING SYSTEM

- **Limited Real-Time Availability Updates** – Many systems do not update appointment slots in real-time, leading to double bookings and scheduling conflicts.
- **Manual Appointment Handling** – Some salons still rely on manual confirmations, causing delays and inefficiencies in booking management.

- **Weak Security Measures** – Some systems do not have strong encryption and security protocols, making user data vulnerable to breaches.
- **Absence of Detailed Analytics** – Existing systems may not provide comprehensive reports on customer behaviour, sales trends, and appointment frequency.
- **Poor Mobile Optimization** – Some booking platforms are not fully responsive, causing usability issues on mobile devices.
- **Lack of Integrated Product Management** – Many systems do not allow salons to manage beauty product sales along with appointment bookings.
- **Minimal Feedback and Review Management** – Existing solutions may not effectively handle customer reviews and ratings, limiting insights for improvement.

2.4 PROPOSED SYSTEM

The proposed system for the beauty salon booking platform is designed to overcome the limitations of existing systems while enhancing user experience and operational efficiency. It ensures a seamless appointment booking process with real-time availability updates, reducing scheduling conflicts. The platform features an intuitive user interface with easy service navigation, filtering options, and personalized recommendations based on user preferences.

To improve convenience, the system integrates multiple secure payment gateways, allowing users to pay via PayPal, Stripe, credit/debit cards, and digital wallets. Enhanced security measures protect user data and transactions, ensuring compliance with industry standards. The platform also includes beauty product management, enabling salons to list and sell products alongside service bookings.

A mobile-optimized design ensures smooth access across various devices, while automated notifications and reminders enhance customer engagement. Additionally, the system provides robust customer support features, including live chat and feedback management, to promptly address user concerns. With integrated analytics tools, salon administrators can track booking trends, revenue, and customer behavior for strategic improvements, making the platform more efficient, user-friendly, and business-oriented.

2.5 ADVANTAGES OF PROPOSED SYSTEM

- **Real-Time Appointment Booking** – Eliminates scheduling conflicts with instant availability updates.
- **User-Friendly Interface** – Provides intuitive navigation, service filtering, and a seamless booking process.
- **Enhanced Security Measures** – Implements encryption and data protection to safeguard user information.
- **Mobile Optimization** – Ensures smooth functionality across all devices, improving accessibility.
- **Automated Notifications & Reminders** – Sends alerts for upcoming appointments, offers, and product promotions.
- **Integrated Product Management** – Allows salons to sell beauty products alongside appointment bookings.
- **Comprehensive Analytics** – Tracks user behavior, appointment trends, and revenue for business insights.
- **Efficient Admin Dashboard** – Enables easy management of services, users, payments, and reports.
- **Review & Rating System** – Allows customers to leave feedback, enhancing service quality and trust.
- **Scalability & Flexibility** – Can be expanded to accommodate more salons, services, and advanced features.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDY

The **Serenity Styles** is a web-based platform designed to streamline and enhance the appointment scheduling process at beauty salons. The system offers users a seamless experience by allowing them to create accounts, browse available services, book appointments, and make secure payments via integrated payment gateways. Salon administrators can efficiently manage appointments, services, user accounts, product inventories, and customer feedback through an intuitive Admin Dashboard. This feasibility study evaluates the various dimensions of the project to assess whether it is viable for implementation. The study focuses on the technical, economic, operational aspects to ensure that the system can be developed, deployed, and maintained effectively while meeting market demand.

Types of Feasibility

- Economical feasibility
- Technical feasibility
- Operational feasibility

3.1.1 Economical Feasibility

Economic feasibility refers to the assessment of a project's financial aspects to determine whether it is a worthwhile investment. It involves evaluating the estimated development, implementation, and operational costs against the expected financial benefits or revenue. This analysis helps stakeholders understand the potential return on investment (ROI) and whether the project is sustainable and profitable in the long run. A project is considered economically feasible if it can recover its costs within a reasonable time and generate profit thereafter.

For Serenity Styles, development costs are estimated between ₹10,000 to ₹20,000, with annual maintenance costs of ₹2,000 to ₹5,000. Revenue will come from salon subscriptions, transaction fees, and advertising. With high market demand and a scalable model, the platform is expected to break even within 6 to 12 months, making it economically feasible.

3.1.2 Technical Feasibility

Technical feasibility assesses whether a proposed system can be developed and implemented using existing technologies, tools, and expertise. It involves evaluating the software and hardware requirements, system architecture, scalability, integration capabilities, and security

measures. This analysis helps ensure that the project can be technically executed within the available resources and constraints.

In the case of Serenity Styles, the platform will be built using widely adopted technologies—HTML, CSS, and JavaScript for the front end, and PHP with MySQL for the back end. The system will support real-time appointment booking, secure payment integration via PayPal or Stripe, and responsive design for mobile and desktop access. These technologies are reliable, well-supported, and scalable, making the development of Serenity Styles technically feasible and suitable for future expansion.

3.1.3 Behavioral Feasibility

Behavioral feasibility evaluates how the proposed system will be received by its intended users. It focuses on user interaction, adaptability, satisfaction, and the likelihood of acceptance. This involves assessing whether users can easily learn and use the system, whether the system meets their expectations, and how it will impact their daily routines or operations. A system is considered behaviorally feasible if it enhances user experience and gains acceptance with minimal resistance or training.

In the case of Serenity Styles, the system is designed to meet the expectations of both customers and salon staff. Customers benefit from a simple and intuitive interface to browse services, book appointments, and make secure payments, with added features like real-time availability, notifications, and reviews enhancing satisfaction. Salon administrators have access to a streamlined dashboard to manage appointments, services, and finances efficiently. The platform requires minimal training and is easy to use, ensuring smooth adoption and high user satisfaction, making it behaviorally feasible.

3.1.4 Feasibility Study Questionnaire

1. How would the organization cope if this system was not implemented?

If the Serenity Styles system is not implemented, the organization would continue relying on manual or semi-digital methods to manage appointments, customer information, payments, and service scheduling. This would likely result in ongoing inefficiencies such as overbookings, scheduling errors, lack of proper customer follow-up, and missed business opportunities. The absence of an online platform would also reduce accessibility for tech-savvy customers who prefer booking services digitally. As competitors adopt modern solutions, the salon may fall behind in offering a customer-friendly experience, which could negatively impact growth and customer retention.

2. What are the problems with current processes and how would a new system help alleviate these problems?

The current processes in most beauty salons are often manual or use basic tools like phone calls and notebooks for scheduling, which are prone to human errors and time-consuming. These methods make it difficult to track appointments, handle peak time bookings, manage inventory, and collect customer feedback. The Serenity Styles system addresses these issues by providing a centralized digital platform that automates appointments, tracks product inventory, manages customer profiles, and facilitates secure online payments. It also offers real-time service availability and customer notifications, significantly improving accuracy, efficiency, and customer satisfaction.

3. What direct contribution will the system make to the business objectives and requirements?

The Serenity Styles system directly contributes to the salon's business goals by streamlining operations, enhancing customer experience, and increasing revenue opportunities. It helps reduce manual workload for staff, minimizes appointment errors, and improves customer engagement through a seamless booking process. Features like real-time availability, service reviews, and secure payment gateways not only attract new clients but also help in retaining existing ones. By aligning with the objectives of improved service quality, operational efficiency, and business growth, the system plays a vital role in transforming how the salon operates.

4. Can information be transferred to and from other organizational systems?

Yes, the Serenity Styles system is designed with flexibility and interoperability in mind. It can be integrated with popular payment gateways like PayPal and Stripe for seamless financial transactions. Reports on appointments, customer feedback, and product sales can be exported for use in accounting software or customer relationship management (CRM) tools. This ensures smooth data flow across different departments and systems within the organization, supporting better decision-making and overall business management.

5. Does the system require technology that has not previously been used in the organization?

The implementation of Serenity Styles may involve the adoption of new technologies that the salon hasn't used before, such as online payment gateways, a responsive web-based interface, and a backend database for centralized data storage. Technologies like PHP, MySQL, and Stripe

or PayPal might be new to the organization but are widely used and supported. While a brief learning curve may exist for staff, these technologies are easy to adapt to and provide significant benefits in terms of efficiency, scalability, and customer satisfaction.

6. What must be supported by the system and what need not be supported?

The system must support critical features such as customer registration and login, real-time service booking, appointment management, online payments, service and product listings, notifications, and customer reviews. These functionalities are essential to achieving the platform's purpose and meeting user expectations. On the other hand, features like multi-language support, advanced analytics, integration with external salon hardware (like POS systems), or in-app chat support are not essential in the initial phase and can be considered for future upgrades based on user demand and business growth.

7. Project Overview ?

The Serenity Styles is an online salon booking system that allows users to schedule appointments, purchase beauty products, and make secure payments.

8. To what extent is the system proposed ?

The system is designed to streamline salon appointment management, improve user experience, and enhance operational efficiency.

9. Specify the viewers/public involved in the system ?

- Salon customers
- salon administrator
- Beauty product buyers.

10. List the modules included in your system ?

- User Module
- Admin Module
- Appointment Booking
- Product Management
- Service management
- Payment System

- Ratings & Reviews and Notifications.

11. Identify the users in your project ?

- Salon customers (users)
- Salon administrator (admin).

12. Who owns the system ?

The salon owner implementing the platform.

13. System is related to which firm/industry/organization ?

Beauty and wellness industry.

14. Details of the person you have contacted for data collection ?

Shop Name : Ann Mariya Bridal Studio

Owner Name : Neena Mathew

Place : Ranni,Pathanamthitta

Contact : 9946697126

Questionnaire to collect details about the project

1. **How do you currently manage appointments in your salon?**

   Manually via a paper calendar, leading to double-bookings and inefficiencies.

2. **What challenges do you face with your current appointment scheduling process?**

   Double-bookings, lack of real-time availability, and difficulty in tracking customer preferences.

3. **Would you be interested in a system that integrates both appointment booking and beauty product sales?**

   Yes, it would streamline operations and boost sales.

4. **How important is it for your salon to accept online payments ?**

   Very important, as cashless payments offer convenience and ensure timely transactions.

5. **What features would you prioritize in a salon booking system?**

   Automatic reminders, secure payments, service management, and customer feedback

tracking.

6. **How do you currently handle product inventory management?**

   Manually, which leads to stock issues. A real-time tracking system is needed.

7. **Would you find value in collecting customer feedback for services and products?**

   Yes, it helps improve services, attract new clients, and enhance customer loyalty.

8. **What is your preferred method of customer notification for appointment reminders or promotions?**

   SMS and email.

9. **How much are you willing to pay for a comprehensive booking system with product sales integration?**

   ₹5000-₹10,000 per month, with flexibility for advanced features.

10. **How important is it for your system to provide real-time availability updates?**

    Very important for preventing double-bookings and improving scheduling efficiency.

3.2 SYSTEM SPECIFICATION

3.2.1 HARDWARE SPECIFICATION

Processor - Ryzen i5

RAM - 4 G B

Hard disk - S S D

3.2.2 SOFTWARE SPECIFICATION

Front End	-	HTML, CSS
Back End	-	PHP, JavaScript
Database	-	MYSQL
Client on PC	-	Windows 7 and above.
Technologies used	-	JS, HTML5, AJAX, J Query, PHP, CSS

3.3 SOFTWARE DESCRIPTION

3.3.1 PHP

PHP (Hypertext Preprocessor) is a popular open-source server-side scripting language used for web development. It integrates seamlessly with HTML and supports various databases like MySQL and PostgreSQL, enabling the creation of dynamic and interactive websites. PHP is widely adopted due to its cross-platform compatibility, ease of use, and strong community support. It powers many content management systems (CMS) like WordPress and frameworks like Laravel. With built-in security features, database integration, and scalability, PHP remains a preferred choice for developing e-commerce sites, web applications, and API services.

3.3.2 MySQL

SQL (Structured Query Language) is a standardized programming language used to manage and manipulate relational databases. It enables users to create, read, update, and delete (CRUD) data efficiently. SQL supports various database management systems (DBMS) like MySQL, PostgreSQL, SQL Server, and Oracle. It provides powerful features such as data querying, indexing, transaction control, and security management. With its structured syntax and scalability, SQL is widely used in web applications, enterprise systems, and data analytics to ensure efficient data storage and retrieval.

CHAPTER 4

SYSTEM DESIGN

4.1 INTRODUCTION

The design phase is a crucial stage in the development of the Serenity Styles beauty salon booking system, as it lays the foundation for an efficient and user-friendly platform. A well-structured design ensures smooth functionality, optimized performance, and a seamless user experience. This phase involves defining the system architecture, database structure, user interface, and integration of various components, such as appointment scheduling, product management, and secure payment processing. The design process translates user requirements into a detailed technical blueprint, enabling developers to implement a robust and scalable solution. By prioritizing usability, security, and responsiveness, the system is crafted to enhance both customer and salon management experiences, making booking and service management more efficient and hassle-free.

4.2 UML DIAGRAM

A standardized dialect called Unified Modeling Language (UML) is utilized to conceptualize, characterize, plan, and depict program frameworks. The Question Administration Gather (OMG) was dependable for creating UML, and the primary draft of the UML 1.0 definition was discharged in January 1997. Programming dialects like Java, C++, and COBOL are not the same as UML. It could be a nonexclusive visual demonstrating dialect utilized for computer program frameworks and a pictorial dialect utilized for program outlines. UML may be utilized for non-software frameworks, such as fabricating forms, indeed though it is generally utilized to speak to program frameworks.

- Class diagram
- Object diagram
- Use case diagram
- Sequence diagram
- Collaboration diagram
- Activity diagram
- State chart diagram
- Deployment diagram
- Component diagram

4.2.1 Use Case Diagram

A use case diagram is a graphical representation that illustrates how users and external entities interact with a system. It helps identify, outline, and organize the system's functional requirements from a user's perspective. Built using the Unified Modeling Language (UML), use case diagrams define system boundaries, actors (users or external systems), use cases (specific actions or functionalities), and their relationships. They are essential for understanding system behavior, verifying requirements, testing, and providing customer support. To create an effective use case diagram, it is important to use clear naming conventions, accurately depict actor roles, highlight essential functionalities, and avoid unnecessary complexity. Following these principles ensures a well-structured diagram that effectively communicates the system's core functionalities.

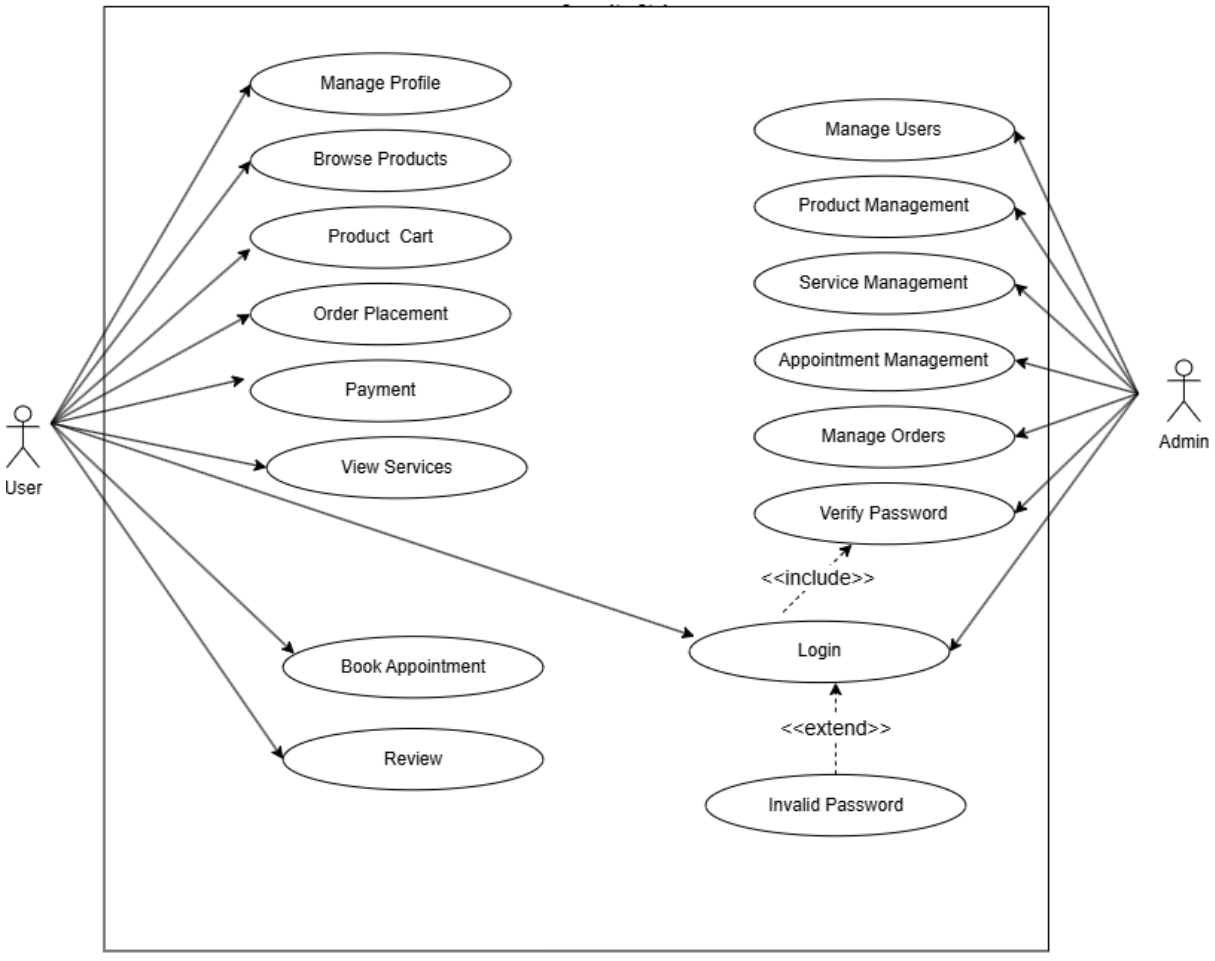


Figure 1 : Use case diagram for Serenity Styles

4.2.2 Sequence Diagram

A sequence diagram is a type of interaction diagram that depicts the chronological flow of messages between system components. It helps visualize object interactions, understand system requirements, and detect potential issues. Key elements include actors (external users or systems), lifelines (vertical dashed lines representing an object's lifespan), messages (interactions between objects), and guards (conditions that control message flow). Sequence diagrams are used to model complex processes, illustrate use case details, and analyze system functionality. They aid both developers and business professionals in understanding and refining system behavior effectively

Uses of sequence diagram

- Modeling and visualizing the logic of complex functions, operations, or procedures.
- Showing details of UML use case diagrams.
- Understanding the detailed functionality of current or future systems.
- Visualizing how messages and tasks move between objects or components in a system.
- Overall, sequence diagrams are useful for representing the flow of interactions.

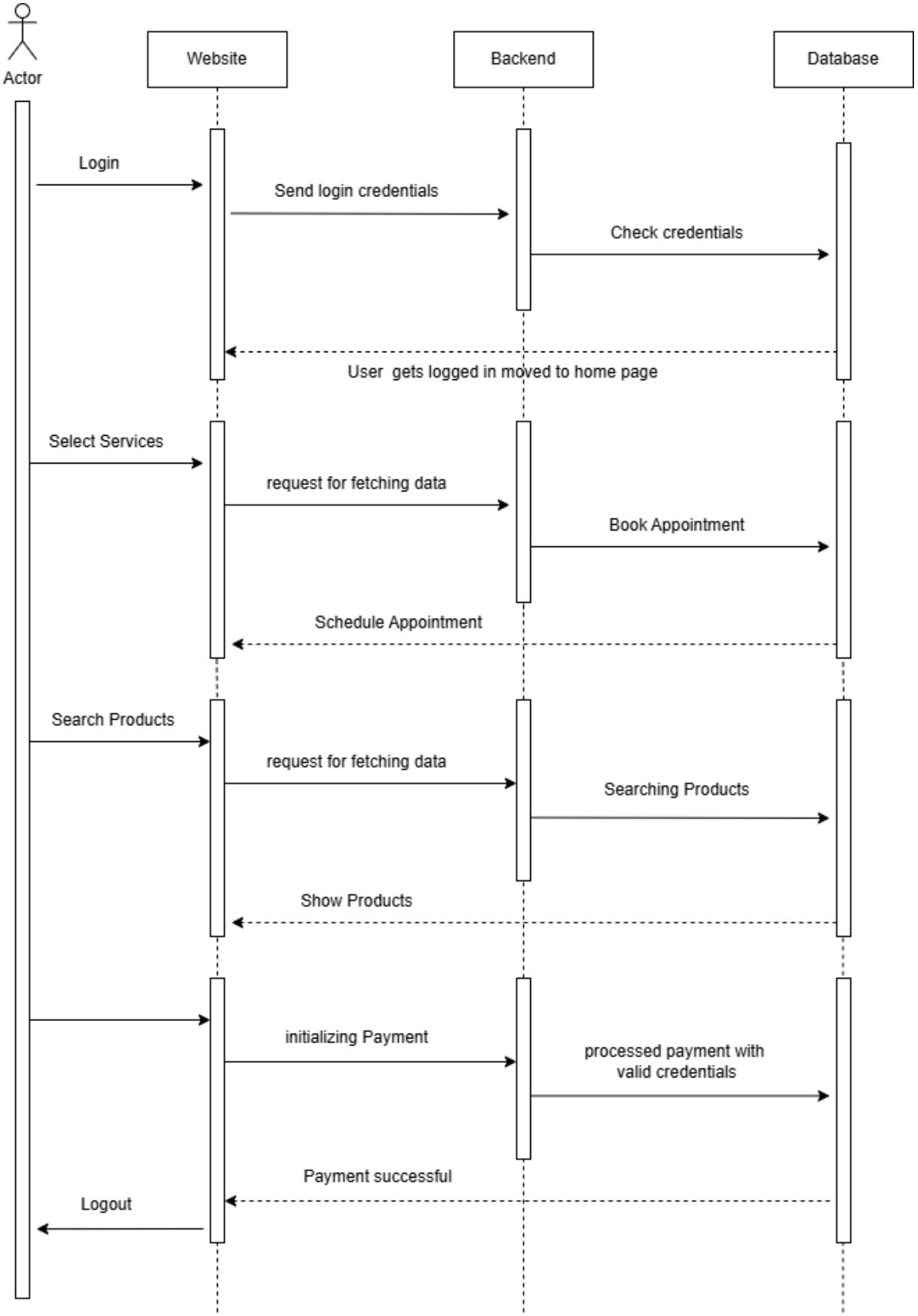


Figure 2 : Sequence diagram for Serenity Styles

4.2.3 State Chart Diagram

A State Chart Diagram is a UML behavioral diagram that represents the various states an object or system can be in and how it transitions between them. It consists of key elements like the Initial State (starting point), State (current condition of an object), Transition (movement between states), Event and Action (triggers and responses), Signal (messages causing transitions), and the Final State (end of the process). State diagrams help visualize system behavior over time, making them useful for modeling dynamic systems and workflow processes.

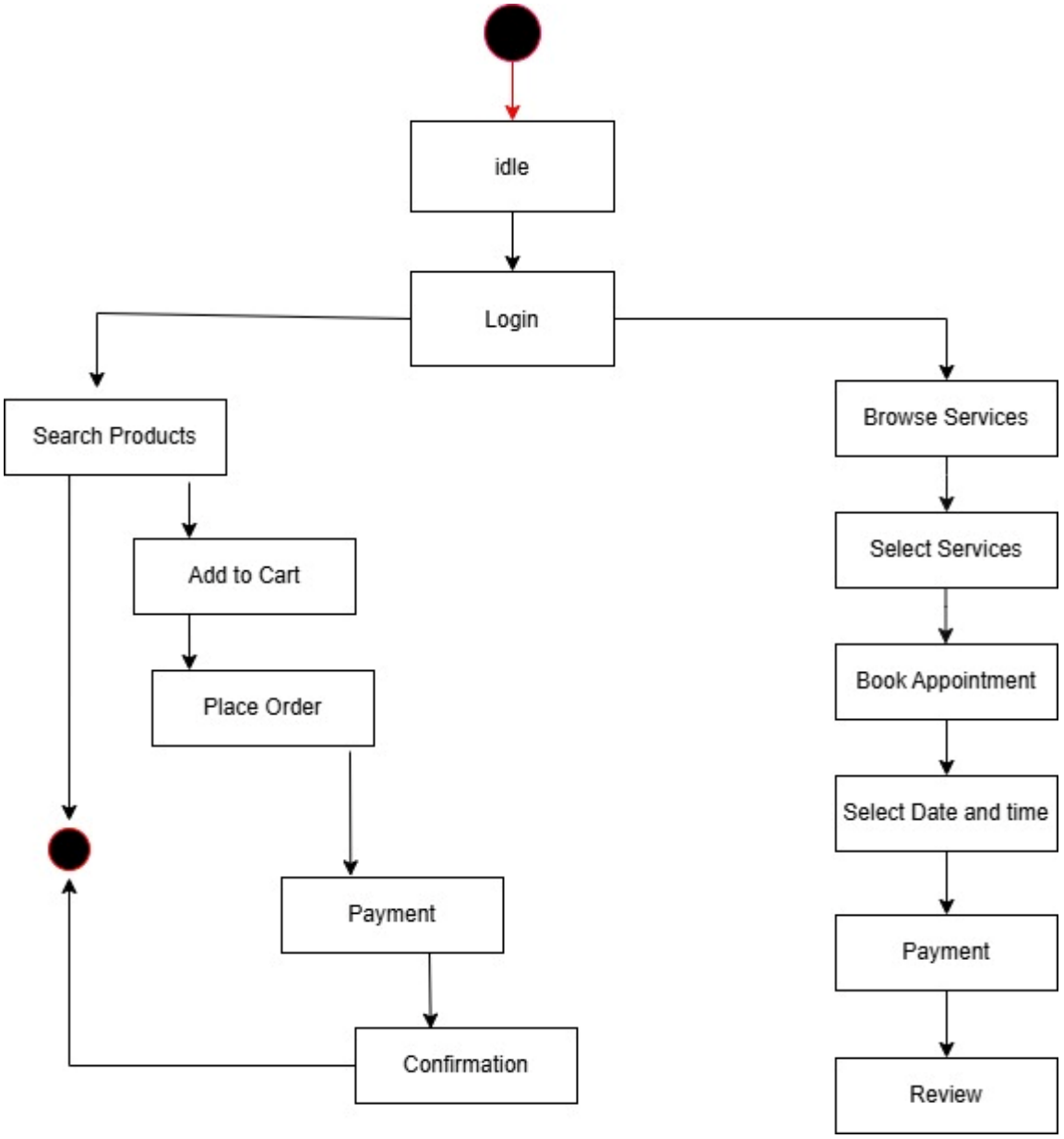


Figure 3 : State Chart diagram for Serenity Styles

4.2.4 Activity Diagram

An activity diagram is a visual representation of a workflow that shows how one activity leads to another. An activity is referred to as a system operation, and one operation leads to another in the control flow. A flow can be parallel, concurrent, or branched, and activity diagrams use various functions such as branching, joining, etc., to manage all types of flow control. Activity diagrams are a type of behavior diagram that shows the behavior of a system. They show the flow of control from the start point to the end point and show the different decision paths that exist during the execution of the activity.

The key components of an activity diagram are:

- **Initial node**: A starting point of the activity diagram, denoted by a black circle.
- **Activity**: A task or action performed by the system or entity, represented by a rectangle with rounded corners.
- **Control flow**: It represents the sequence of activities or actions performed by the system or entity, represented by an arrow.
- **Decision node**: A decision or branching point in the activity flow, denoted by a diamond shape.
- **Merge node**: Used to merge multiple branches of the activity flow into a single flow, represented by a diamond shape with a plus sign inside.
- **Fork node**: Used to split the activity flow into multiple parallel flows, represented by a solid black circle with multiple arrows.
- **Join node**: Used to join multiple parallel flows back into a single flow, represented by a solid black circle with multiple arrows pointing towards it.
- **Final node**: The end point of the activity diagram, denoted by a black circle with a dot inside.
- **Object flow**: Represents the flow of objects or data between activities, represented by a dashed arrow.

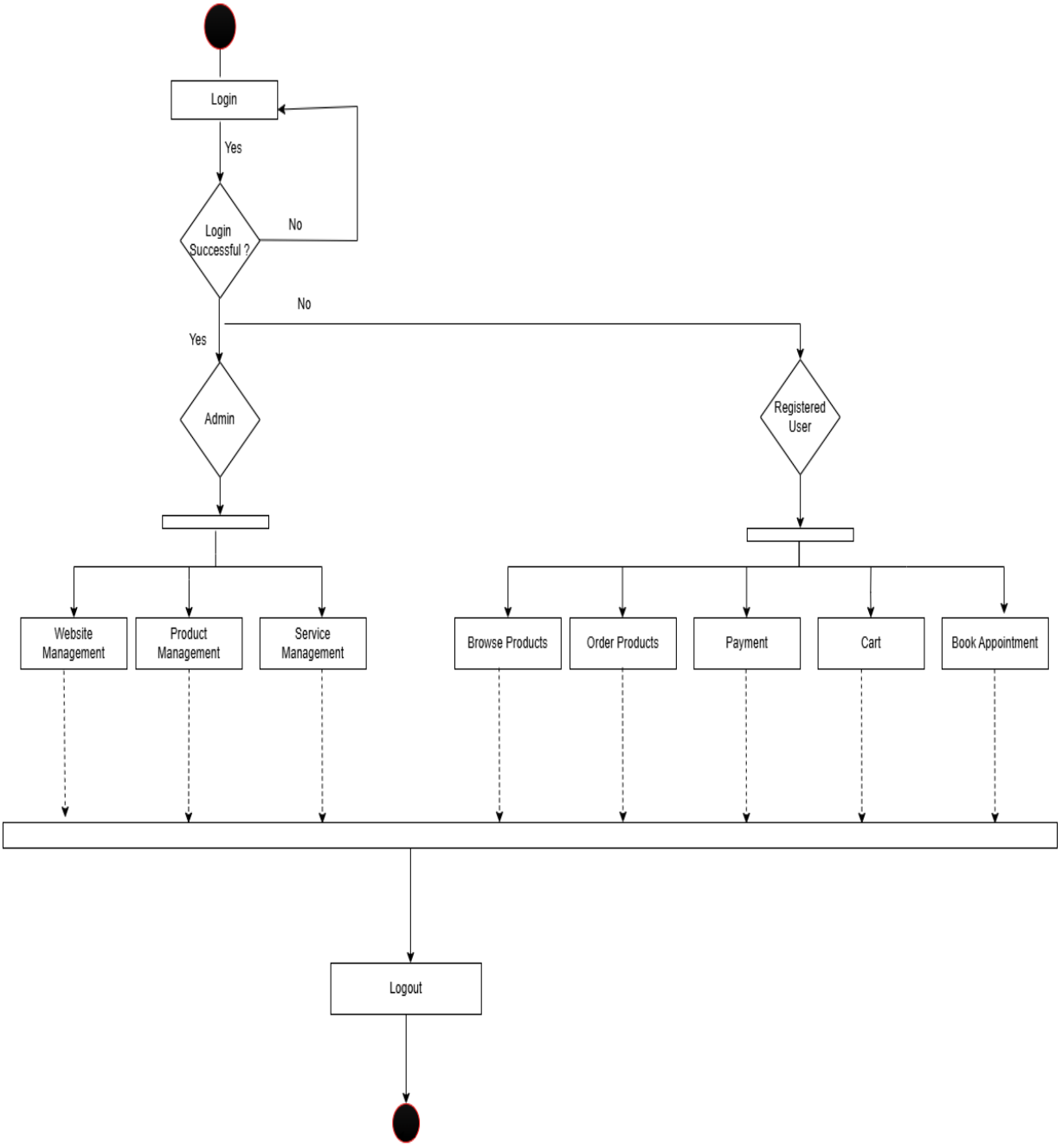


Figure 4 : Activity diagram for Serenity Style

4.2.5 Class Diagram

A Class Diagram is a key UML component used in object-oriented modeling to represent the structure of an application. It defines classes, objects, interfaces, and relationships such as association, aggregation, composition, inheritance, dependency, and multiplicity. Classes are depicted as rectangles containing attributes and methods, while interfaces define abstract methods. Relationships like aggregation (whole-part), composition (stronger whole-part), and inheritance (is-a hierarchy) establish connections between classes. Class diagrams provide a conceptual and detailed view of system structure, making them essential for both software development and data modeling.

Important Components of a Class Diagram:

- **Class:** A blueprint for objects, represented as a rectangle with a class name, attributes, and methods.
- **Interface:** Defines abstract methods, represented as a circle with the interface name.
- **Object:** An instance of a class with state and behavior, shown as a rectangle.
- **Association:** A link between two classes, depicted as a line with optional direction and multiplicity.
- **Aggregation:** A part-whole relationship, shown with a hollow diamond at the whole side.
- **Composition:** A stronger part-whole relationship, represented with a filled diamond.
- **Inheritance:** An "is-a" relationship between superclass and subclass, marked by an open arrow.
- **Dependency:** One class depends on another, shown with a dashed arrow.

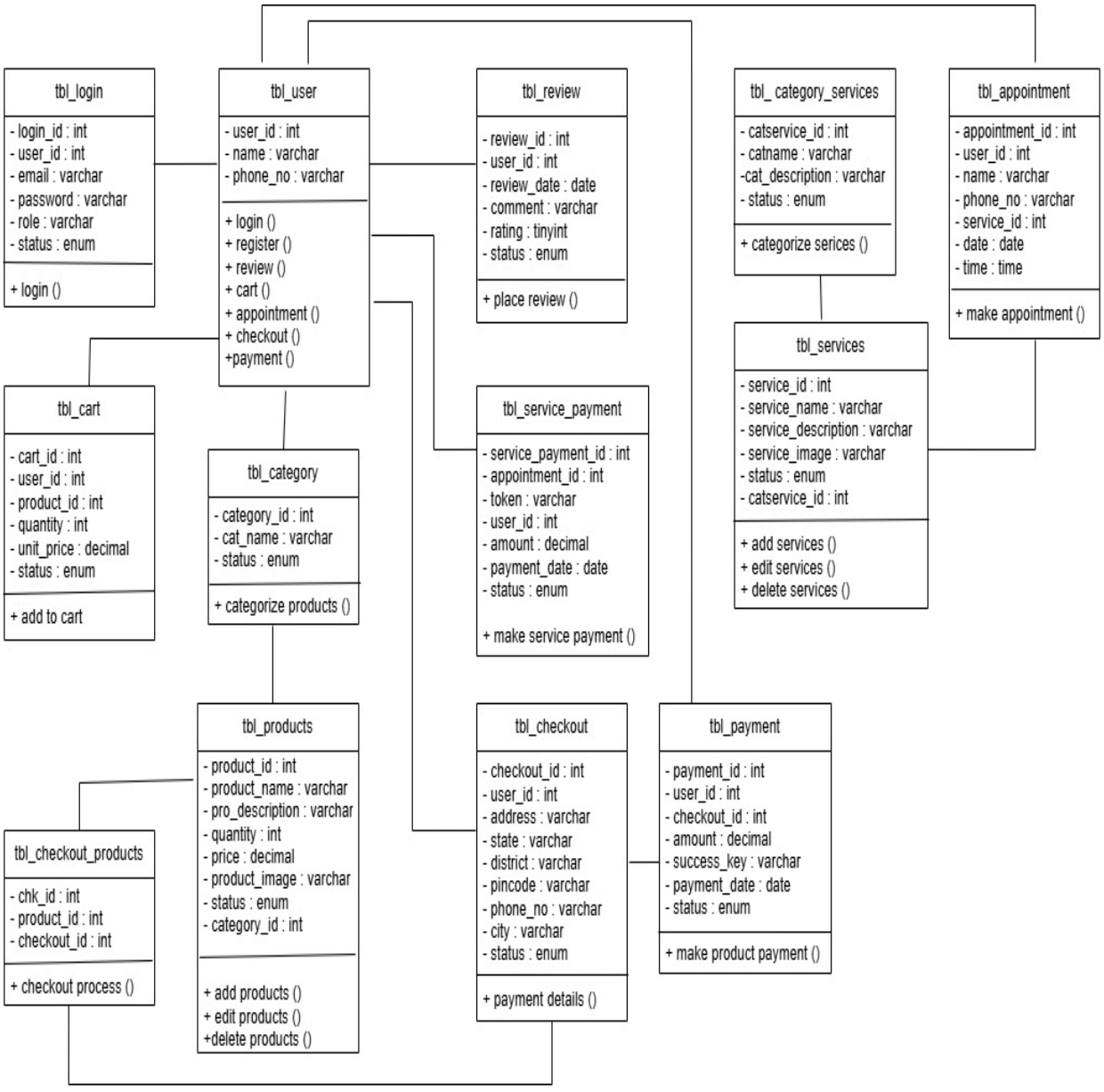


Figure 5 : Class diagram for Serenity Styles

4.2.6 Object Diagram

Class diagrams and object diagrams are closely related in object-oriented modeling. Object diagrams are instances of class diagrams, which represent a snapshot of the system at a given moment in time. Both types of diagrams use the same concepts and notation to represent the structure of a system. While class diagrams are used to model the structure of the system, including its classes, attributes, and methods, object diagrams represent a group of objects and their connections at a specific point in time. An object diagram is a type of structural diagram in UML that shows instances of classes and their relationships.

Key Components of an Object Diagram:

- **Object:** An instance of a class representing a specific entity, shown as a rectangle.
- **Class:** A template for objects with attributes and methods, represented as a three-part rectangle.
- **Link:** A relationship between objects, depicted as a line.
- **Attribute:** A property defining an object's state, shown as a name-value pair.
- **Value:** A specific instance of an attribute.
- **Operation:** An action an object can perform, listed inside the class rectangle.
- **Multiplicity:** Defines the number of instances in a relationship.

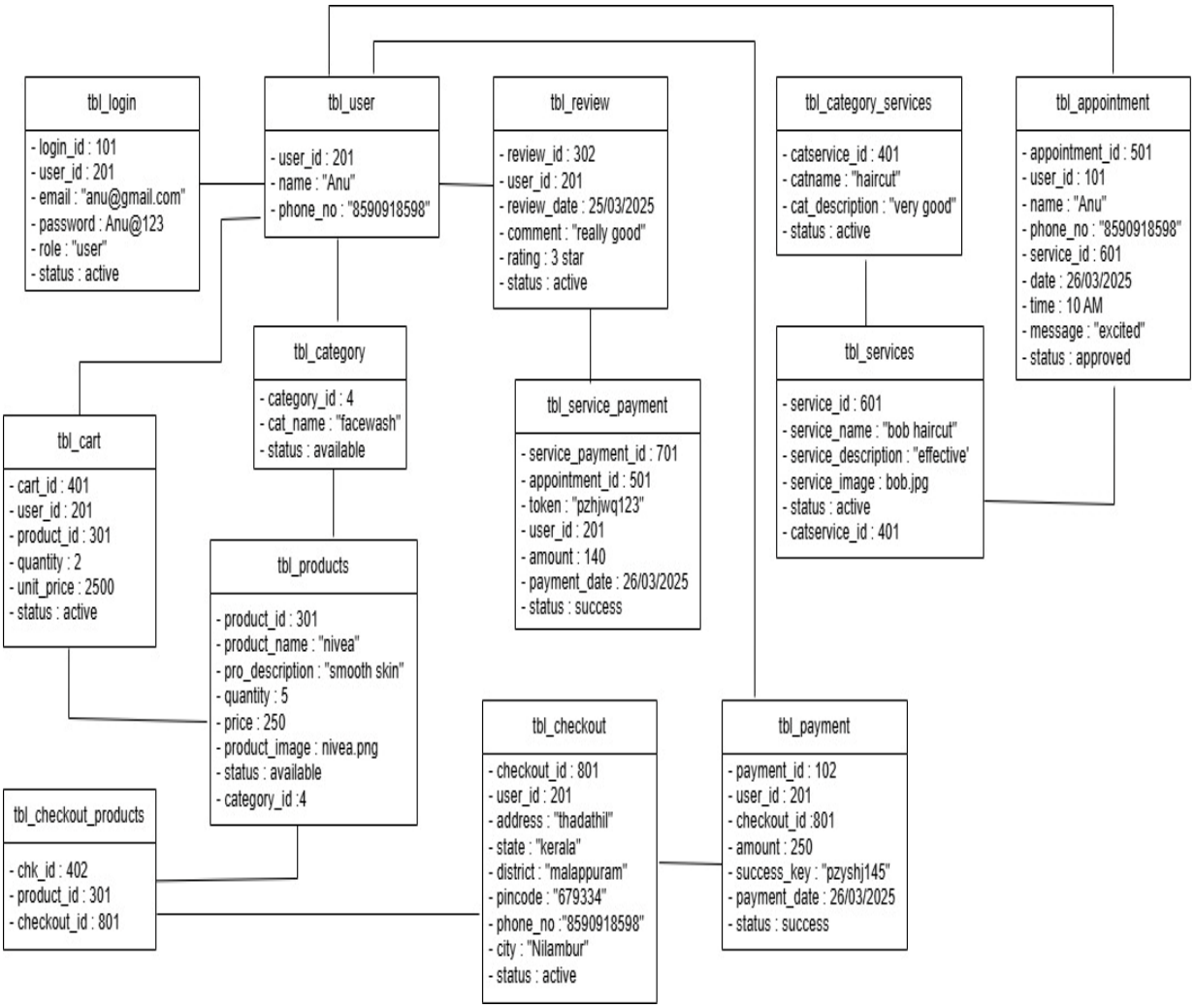


Figure 6 : Object diagram for Serenity Styles

4.2.7 Component Diagram

A component diagram in UML illustrates how various components are interconnected to create larger components or software systems. It is an effective tool for representing the structure of complex systems with multiple components. By using component diagrams, developers can easily visualize the internal structure of a software system and understand how different components work together to accomplish a specific task.

Key Components of a Component Diagram:

- **Component**: A modular unit with defined functionality, shown as a rectangle.
- **Interface**: A set of accessible methods, represented as a circle.
- **Port**: A connection point for interactions, shown as a small square.
- **Connector**: A link enabling communication, depicted as a line.
- **Dependency**: A relationship where one component relies on another, shown as a dashed arrow.
- **Association**: A connection between components, represented as a line.

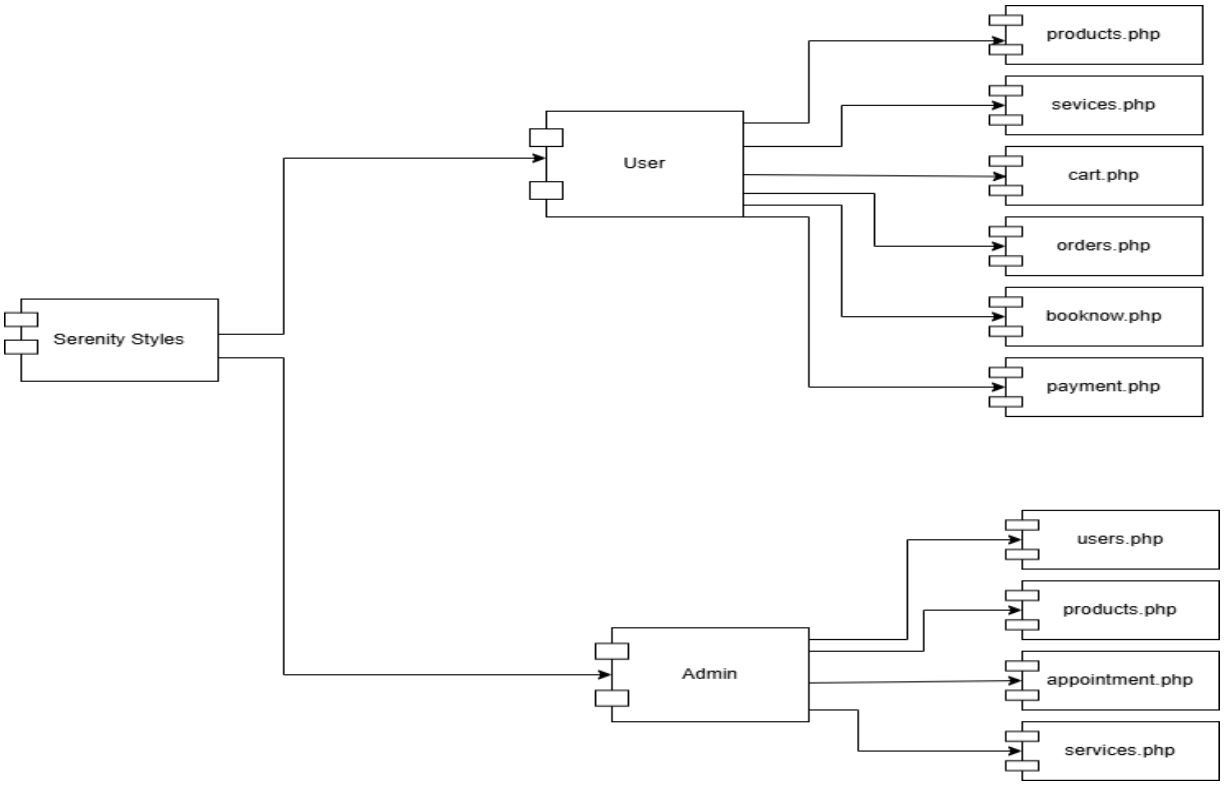


Figure 7 : Component diagram for Serenity Styles

4.2.8 Deployment Diagram

A deployment diagram is a type of UML diagram that focuses on the physical hardware used to deploy software. It provides a static view of a system's deployment and involves nodes and their relationships. The deployment diagram maps the software architecture to the physical system architecture, showing how the software will be executed on nodes. Communication paths are used to illustrate the relationships between the nodes. Unlike other UML diagram types, which focus on the logical components of a system, the deployment diagram emphasizes the hardware topology.

Key Components of a Deployment Diagram:

- **Node**: A physical or virtual machine hosting components, shown as a box.
- **Component**: A software element providing functionality, represented as a rectangle.
- **Artifact**: A physical data piece used or produced by a component.
- **Deployment Specification**: Details how a component or artifact is deployed, including location and configuration.
- **Association**: A relationship between nodes and components, shown as a connecting line.
- **Communication Path**: A network or communication link between nodes, represented by a labeled line.

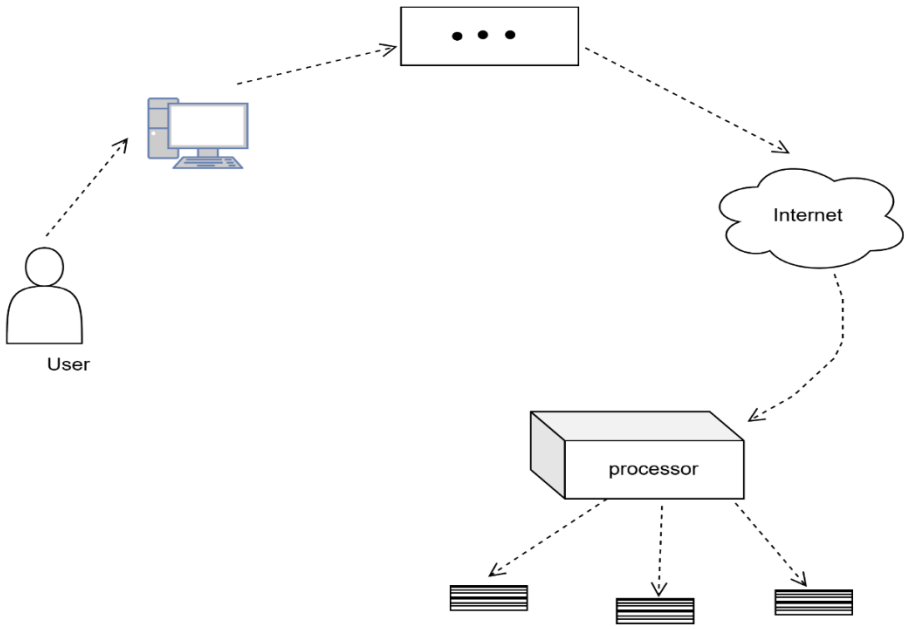


Figure 8 : Deployment diagram for Serenity Styles

4.3 USER INTERFACE DESIGN USING FIGMA

Form Name: Sign Up

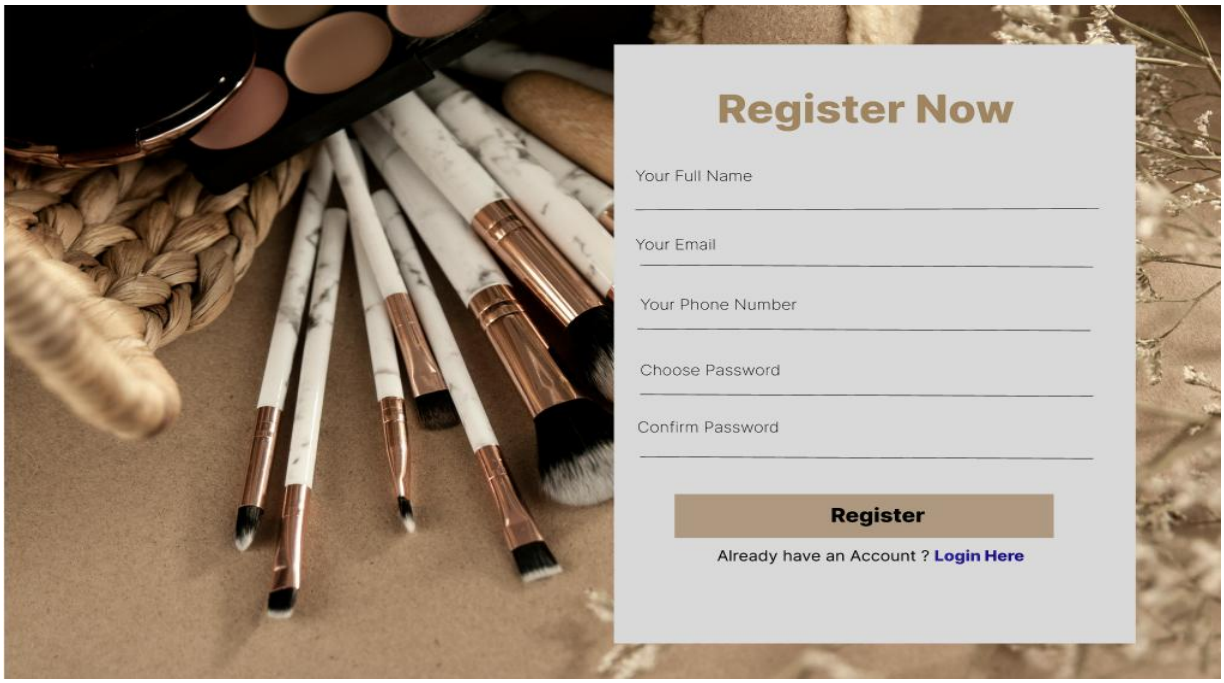


Figure 9 : Sign Up Page

Form Name: Login

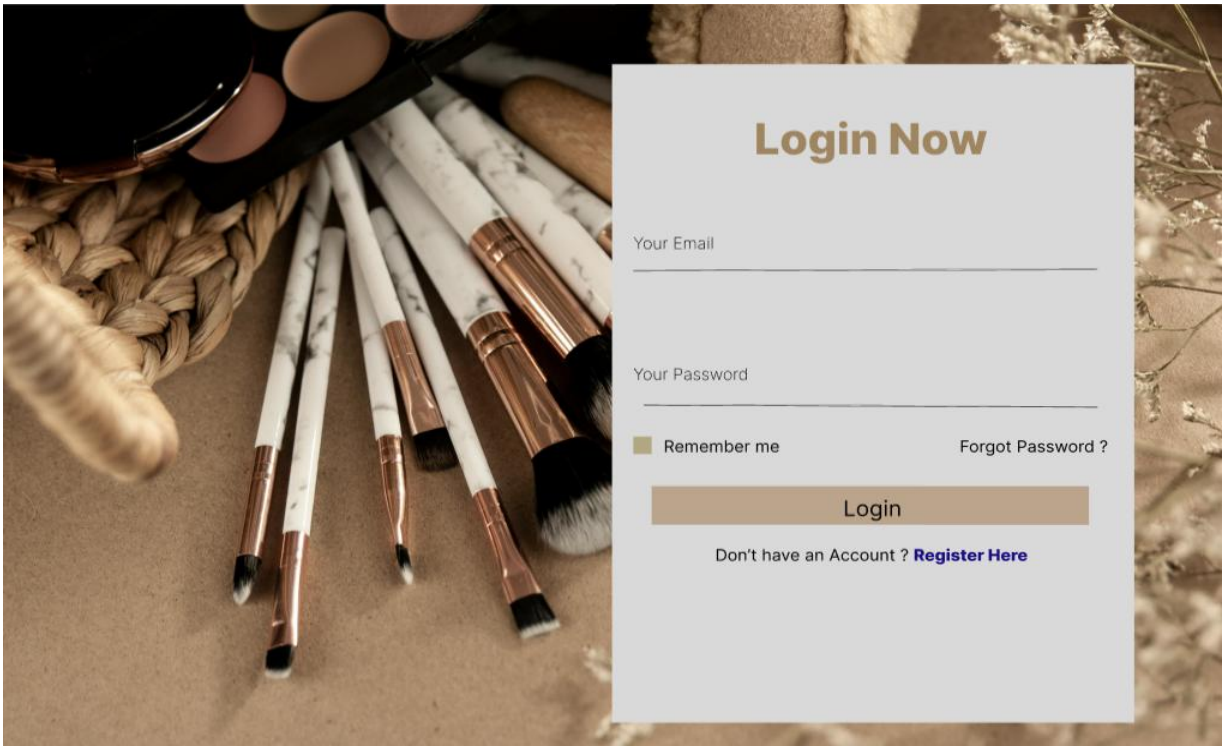


Figure 10 : Login Page

Form Name: User Dashboard

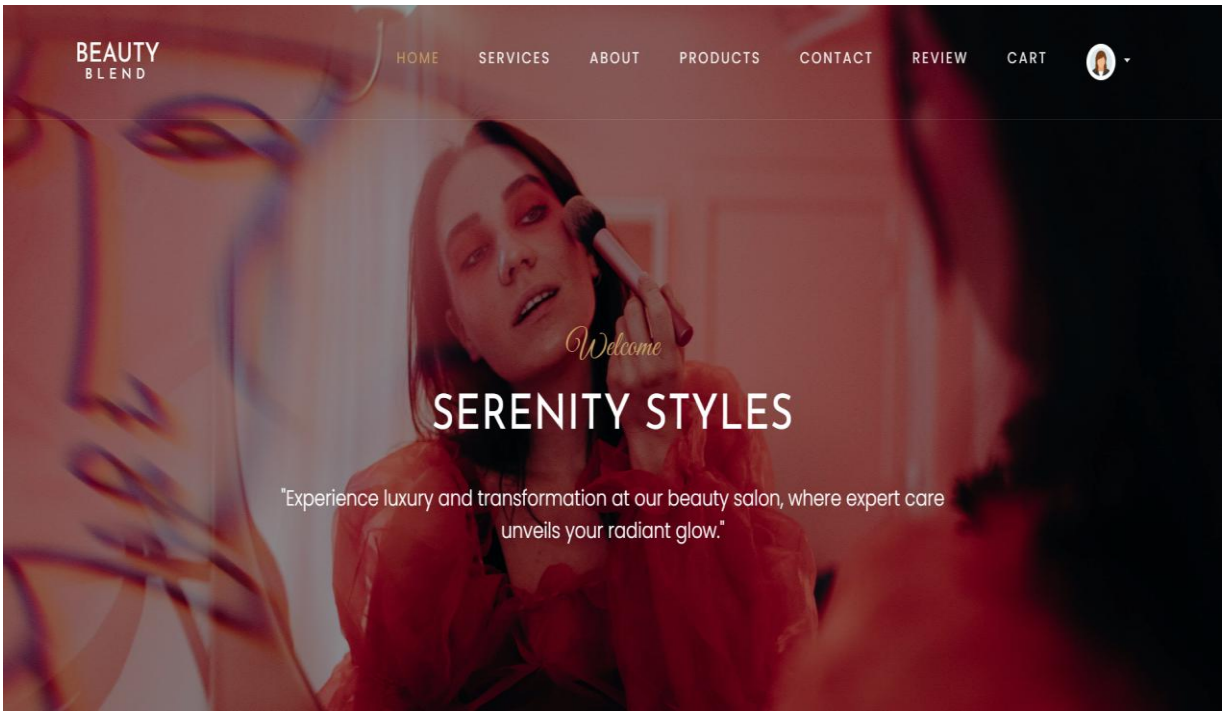


Figure 11 : User Dashboard Page

Form Name: Admin Dashboard

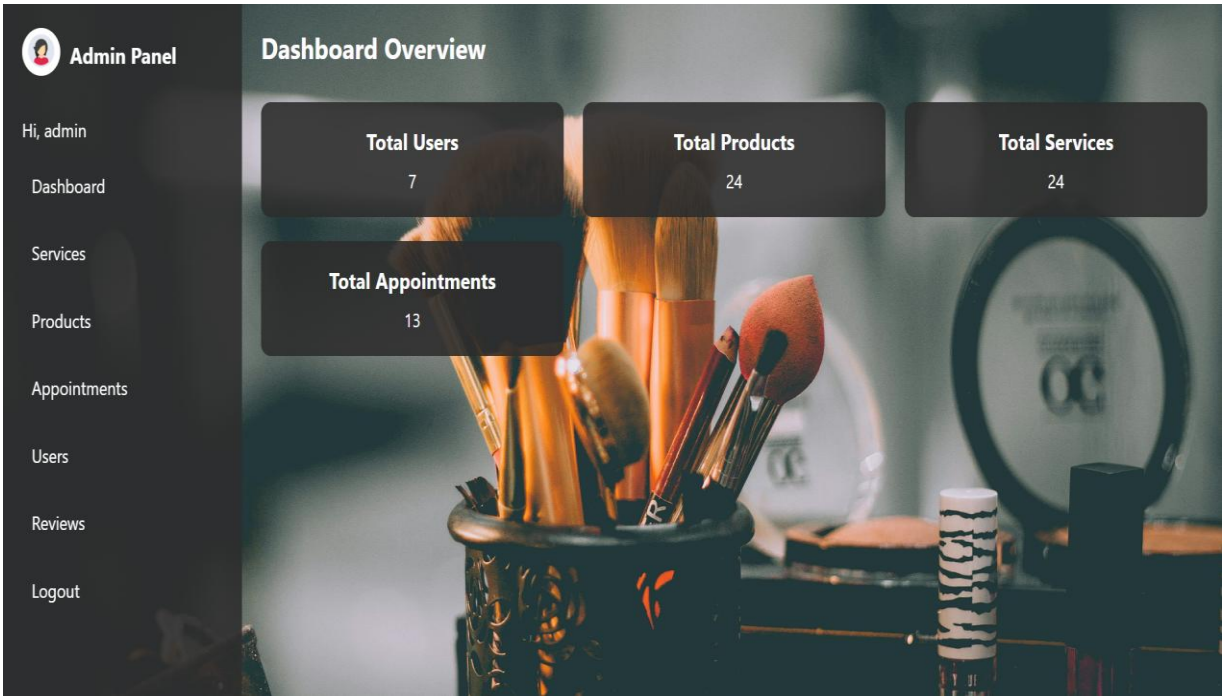


Figure 12 : Admin Dashboard Page

Form Name: Manage Services

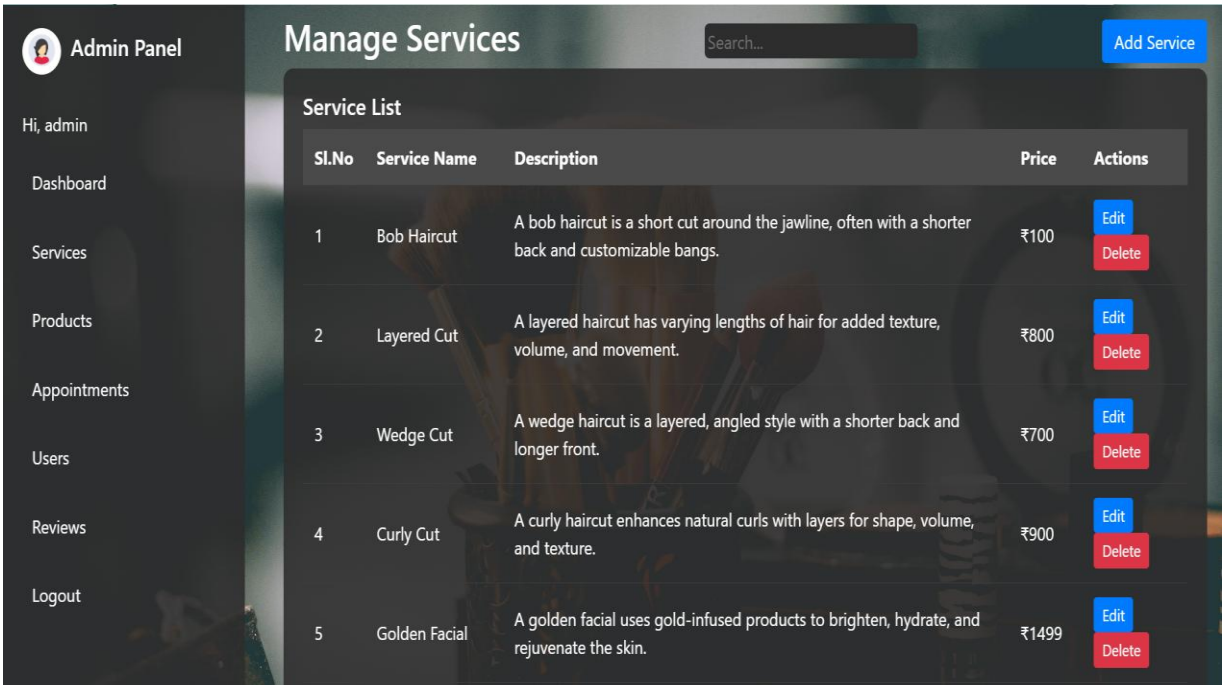


Figure 13 : Manage Services Page

Form Name: Forgot Password

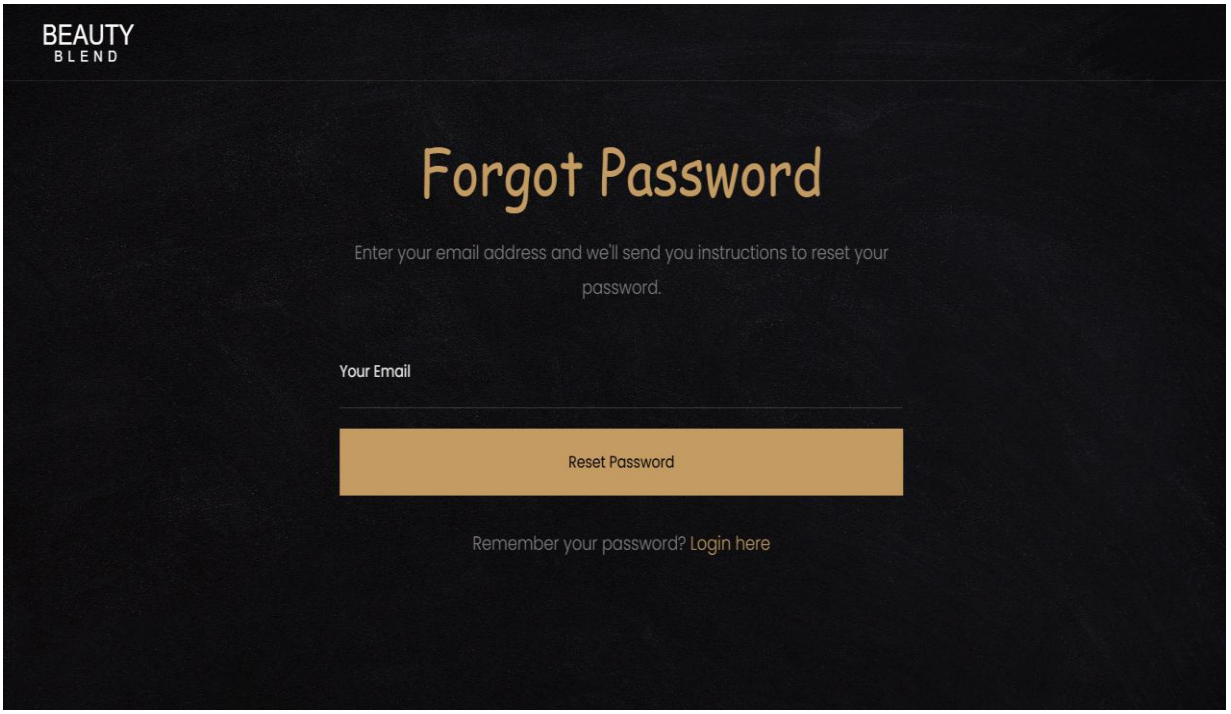


Figure 14: Forgot Password Page

4.4 DATABASE DESIGN

A database is an organized collection of information that's organized to enable easy accessibility, administration, and overhauls. The security of information could be a essential objective of any database. The database design process comprises of two stages. In the first stage, user requirements are gathered to create a database that meets those requirements as clearly as possible. This is known as information-level design and is carried out independently of any DBMS. In the second stage, the design is converted from an information-level design to a specific DBMS design that will be used to construct the system. This stage is known as physical-level design, where the characteristics of the specific DBMS are considered. Alongside system design, there is also database design, which aims to achieve two main goals that are data integrity and data independence.

4.4.1 Relational Database Management System (RDBMS)

An RDBMS (Relational Database Management System) organizes data into tables with rows (records) and columns (attributes). Relationships between tables ensure data integrity, using primary keys (unique identifiers) and foreign keys (linking references). Entity integrity prevents null primary keys, while referential integrity ensures foreign keys match valid primary keys. Domains define data types, and other keys like super keys and candidate keys help maintain structure.

4.4.2 Normalization

Normalization is the process of organizing a database to reduce redundancy and improve data integrity. It divides large tables into smaller ones and establishes relationships using primary keys (unique identifiers) and foreign keys (references to other tables). This process prevents anomalies in insertion, deletion, and updates. The main steps include structuring data, naming tables and columns appropriately, and ensuring proper relationships, resulting in an efficient and manageable database**.**

First Normal Form

A table is in First Normal Form (1NF) if:

- All values in each column are atomic (indivisible).
- Each column contains only a single type of data.
- Each row is uniquely identifiable, meaning there is a primary key.

Example (Before 1NF - Unnormalized Table):

Student_ID	Student_Name	Courses	Contact
101	Alice	Math, Science	9876543210
102	Bob	English	8765432109
103	Charlie	Science, History	7654321098

Conversion to 1NF:

Student_ID	Student_Name	Course	Contact
101	Alice	Math	9876543210
101	Alice	Science	9876543210
102	Bob	English	8765432109
103	Charlie	Science	7654321098
103	Charlie	History	7654321098

Second Normal Form (2NF)

A table is in **Second Normal Form (2NF)** if:

- It is already in 1NF.
- It removes partial dependencies, meaning all non-key attributes must be fully dependent on the entire primary key.
- If the table has a composite primary key, non-key attributes should not be dependent on just a part of the primary key.

Example (Before 2NF - 1NF Table):

Student_ID	Student_Name	Course	Contact
101	Alice	Math	9876543210
101	Alice	Science	9876543210
102	Bob	English	8765432109
103	Charlie	Science	7654321098
103	Charlie	History	7654321098

Conversion to 2NF:

Students Table

Student_ID	Student_Name	Contact
101	Alice	9876543210
102	Bob	8765432109
103	Charlie	7654321098

Enrollment Table

Student_ID	Course
101	Math
101	Science
102	English
103	Science
103	History

Third Normal Form (3NF)

A table is in Third Normal Form (3NF) if:

- It is already in 2NF.
- It removes transitive dependencies, meaning non-key attributes should not depend on other non-key attributes.
- All attributes must be directly dependent on the primary key.

Example (Before 3NF - 2NF Tables):

Students Table (Before 3NF)

Student_ID	Student_Name	Contact	Address
101	Alice	9876543210	New York
102	Bob	8765432109	Los Angeles
103	Charlie	7654321098	Chicago

Enrollment Table

Student_ID	Course
101	Math
101	Science
102	English
103	Science
103	History

Conversion to 3NF:

Students Table

Student_ID	Student_Name
101	Alice
102	Bob
103	Charlie

Contact Table

Contact_ID	Student_ID	Phone	Address
1	101	9876543210	New York
2	102	8765432109	Los Angeles
3	103	7654321098	Chicago

Enrollment Table

Student_ID	Course
101	Math
101	Science
102	English
103	Science
103	History

4.4.3 Sanitization

Data sanitization removes illegal characters or values from input to prevent security risks in web applications. PHP provides a filter extension for sanitizing and validating data like emails, URLs, and IP addresses. Web applications receive input from forms, cookies, APIs, and databases, making sanitization essential to prevent malicious code. Web applications often receive external input from various sources, including user input from forms, cookies, web services data, server variables, and database query results. It is important to sanitize all external input to ensure that it is safe and does not contain any malicious code or values.

4.4.4 Indexing

An index is a database structure that enhances the speed of table operations. Indexes can be created on one or more columns to facilitate quick lookups and efficient ordering of records. When creating an index, it's important to consider which columns will be used in SQL queries and to create one or more indexes on those columns.

In practice, indexes are a type of table that store a primary key or index field and a pointer to each record in the actual table. Indexes are invisible to users and are only used by the database search engine to quickly locate records. The CREATE INDEX statement is used to create indexes in tables.

When tables have indexes, the INSERT and UPDATE statements take longer because the database needs to insert or update the index values as well. However, the SELECT statements become faster on those tables because the index allows the database to locate records more quickly.

4.5 TABLE DESIGN

1.tbl_login

Primary key: **login_id**

Foreign key: **user_id** references table **tbl_user**

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	login_id	Int(11)	Primary key, Auto_increment	Unique identifier for login.
2	email	Varchar(255)	Unique,Not Null	User's email address.
3	password	Varchar(255)	Not Null	Encrypted Password
4	status	Enum	Not Null	Used to identify the user is Active or Inactive.
5	user_id	Int(11)	Foreign Key	References id of tbl_user.
6	role	enum	Not Null	Used to identify the user is an admin, or customer.

2.tbl_user

Primary key: **user_id**

Foreign key: Nil

No:	Field name	Datatype (Size)	Key Constraints	Description of the field
1	user_id	int(11)	Primary Key, Auto_increment	Unique Identifier for users.
2	name	Varchar(255)	Not Null	User's name
3	phone_no	Varchar(15)	Not Null	User's phone number

3.tbl_category_services

Primary key: **catservice_id**

Foreign key: Nil

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	catservice_id	Int(11)	Primary Key, Auto_increment	Unique identifier for category
2	cat_name	Varchar(255)	Not Null	Category name
3	cat_description	Varchar(255)	Not Null	Category description
4	status	Enum	Not Null	Used to identify the category is Active or Inactive.

4.tbl_services

Primary key: **service_id**

Foreign key: **catservice_id** references table **tbl_category_services**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	service_id	Int(11)	Primary Key, Auto_increment	Unique identifier for services
2	service_name	Varchar(255)	Not Null	Service name
3	service_description	Varchar(255)	Not Null	Service description
4	price	Decimal(10,2)	Not Null	Price of the service
5	service_image	Varchar(255)	Not Null	Service image
6	status	enum	Not Null	Used to identify the category is Active or Inactive.
7	catservice_id	Int(11)	Foreign Key	References id of tbl_category_services

5. tbl_payment

Primary key: **payment_id**

Foreign key: **user_id** references table **tbl_user**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	payment_id	Int(11)	Primary Key, Auto_increment	Unique identifier for payment.
2	user_id	Int(11)	Foreign Key	References user_id of tbl_user.
3	checkout_id	Int(11)	Foreign Key	References checkout_id of tbl_checkout.
4	amount	Decimal(10,2)	Not Null	Payment amount
5	success_key	Varchar(255)	Not Null	Unique payment key
6	payment_date	Timestamp	Not Null	Payment date
7	status	Enum	Not Null	Used to identify the payment is Success or Failed.

6. tbl_category

Primary key: **category_id**

Foreign key: Nil

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	category_id	Int(11)	Primary Key, Auto_increment	Unique identifier for category
2	category_name	Varchar(255)	Not Null	Product Category name
3	status	enum	Not Null	Used to identify the category is Available or Unavailable.

7. tbl_appointment

Primary key: **appointment_id**

Foreign key: **user_id** references table **tbl_user, service_id** references table **tbl_services**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	appointment_id	Int(11)	Primary Key, Auto_increment	Unique identifier for appointment
2	user_id	Int(11)	Foreign Key	References user_id of tbl_user
3	name	Varchar(255)	Not Null	Name of the user
4	phone_no	Varchar(12)	Not Null	Phone number of user
5	service_id	Int(11)	Foreign Key	References service_id of tbl_services
6	date	Date	Not Null	Date of appointment
7	time	Time	Not Null	Time of appointment
8	message	Varchar(255)	Not Null	Message about service
9	status	Enum	Not Null	Used to identify the appointment is Approved or Pending or Cancelled.

8. tbl_products

Primary key: **product_id**

Foreign key: **category_id** references table **tbl_category**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	product_id	Int(11)	Primary Key, Auto_increment	Unique identifier for products
2	product_name	Varchar(255)	Not Null	Product name
3	pro_description	Varchar(255)	Not Null	Product description

4	price	DECIMAL(10,2)	Not Null	Product price
5	product_image	Varchar(255)	Not Null	Product image
6	status	enum	Not Null	Used to identify the products is Available or Unavailable.
7	category_id	Int(11)	Foreign Key	References category_id of tbl_category

9. tbl_review

Primary key: **review_id**

Foreign key: **user_id** references table **tbl_user.**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	review_id	Int(11)	Primary Key, Auto_increment	Unique identifier for review.
2	user_id	Int(11)	Foreign Key	References user_id of tbl_user.
3	review_date	Date	Not Null	Review date
4	comment	Varchar(255)	Not Null	Comment about service
5	rating	Tinyint	Not Null	Rating for the service
6	status	Enum	Not Null	Used to identify the review is Active or Inactive.

10. tbl_checkout

Primary key: **checkout_id**

Foreign key: **user_id** references table **tbl_user.**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	checkout_id	Int(11)	Primary Key, Auto_increment	Unique identifier for checkout.
2	user_id	Int(11)	Foreign Key	References user_id of tbl_user.
3	address	Varchar(255)	Not Null	Address of the user.

4	state	Varchar(255)	Not Null	State of the user.
5	district	Varchar(255)	Not Null	District of the user
6	pincode	Int(11)	Not Null	Pincode of the user
7	phone_no	Varchar(12)	Not Null	users Phone number.
8	city	Varchar(255)	Not Null	City of the user
9	status	Enum	Not Null	Used to identify the checkout is Active or Inactive.

11. tbl_cart

Primary key: **cart_id**

Foreign key: **user_id** references table **tbl_user, product_id** references table **tbl_products.**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	cart_id	Int(11)	Primary Key, Auto_increment	Unique identifier for Cart.
2	user_id	Int(11)	Foreign Key	References user_id of tbl_user.
3	product_id	Int(11)	Foreign Key	References product_id of tbl_products.
4	quantity	Int(11)	Not Null	Quantity of products.
5	unit_price	Decimal(10,2)	Not Null	Price of product
6	status	Enum	Not Null	Used to identify the checkout is Active or Inactive.

12. tbl_checkout_products

Primary key: **chk_id**

Foreign key: **product_id** references table **tbl_products, checkout_id** references table **tbl_checkout.**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	chk_id	Int(11)	Primary Key, Auto_increment	Unique identifier for Cart.

2	product_id	Int(11)	Foreign Key	References product_id of tbl_products.
3	checkout_id	Int(11)	Foreign Key	References checkout_id of tbl_checkout.
4	quantity	Int(11)	Not Null	Product Quantity

13. tbl_service _payment

Primary key: **service_pay_id**

Foreign key: **user_id** references table **tbl_user, appointment_id** references table **tbl_appointment.**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	service_pay_id	Int(11)	Primary Key, Auto_increment	Unique identifier for service payment.
2	appointment_id	Int(11)	Foreign Key	References appointment_id of tbl_appointment.
3	token	Varchar(255)	Not Null	Payment key
4	user_id	Int(11)	Foreign Key	References user_id of tbl_user.
5	amount	Decimal(10,2)	Not Null	Payment amount
6	payment_date	Timestamp	Not Null	Payment date
7	status	Enum	Not Null	Used to identify the service payment is Success or Failed.

14. tbl_refund

Primary key: **refund_id**

Foreign key: **user_id** references table **tbl_user**

No:	Field name	Datatype(size)	Key Constraints	Description of the field
1	refund_id	Int(11)	Primary Key, Auto_increment	Unique identifier for verification.
2	user_id	Varchar(255)	Not Null	User's email address
3	appointment_id	Varchar(11)	Not Null	Otp send to user
4	amount	Enum	Not Null	Used to identify the email is verified or not verified.
5	Created_at	Timestamp	Not Null	Date of verification
6	status	Enum	Not Null	Used to identify the service refund is Success or Failed.

CHAPTER 5

SYSTEM TESTING

5.1 INTRODUCTION

System testing is a critical phase in the software development lifecycle where the entire system is tested as a whole to ensure that it meets the specified requirements. It involves verifying the complete and integrated software application to evaluate its compliance with both functional and non-functional requirements such as performance, security, usability, and reliability.Unlike unit or integration testing, which focus on individual components or interactions between modules, system testing is conducted in an environment that closely resembles the real-world usage scenario. This helps identify issues that might arise when different parts of the system work together.

The primary goal of system testing is to validate the end-to-end behavior of the application from the user's perspective. It acts as a final check before the software is delivered to users, ensuring that all features function correctly, data flows smoothly across modules, and the system responds appropriately under various conditions.

By performing system testing, developers and quality assurance teams can detect critical bugs, performance bottlenecks, and unexpected behavior, ultimately improving software quality and user satisfaction.

5.2 TEST PLAN

A Test Plan is a detailed document that outlines the strategy, scope, objectives, resources, schedule, and activities required to validate the quality and functionality of a software product. It serves as a blueprint for the entire testing process and provides a systematic approach to ensure that the software meets its specified requirements.The Test Plan defines what to test, how to test, when to test, and who will perform the testing. It includes details such as:

- **Test Objectives –** What needs to be achieved through testing.
- **Test Scope –** The features or components to be tested and those that are out of scope.
- **Test Environment –** The hardware, software, tools, and configurations required.
- **Test Deliverables –** Documents and outcomes to be produced during the testing process.
- **Roles and Responsibilities –** Who will perform each testing task.
- **Test Schedule –** Timeline for testing activities.
- **Entry and Exit Criteria –** Conditions that define when to start and stop testing.

5.2.1 Unit Testing

Unit Testing is a software testing technique where individual components or functions of a program called units are tested in isolation to ensure that each part of the code works as expected. A unit is typically the smallest testable part of an application, such as a function, method, or class.

The main objective of unit testing is to verify the correctness of logic, handle edge cases, and catch bugs early in the development process. Unit tests are usually written by developers during or after writing a function, and they are often automated using testing frameworks such as JUnit for Java, pytest for Python, or PHPUnit for PHP.

5.2.2 Integration Testing

Integration Testing is a level of software testing where individual modules or components are combined and tested as a group to verify their interactions and ensure they work together as intended. Unlike unit testing, which focuses on testing isolated components, integration testing checks how modules communicate with each other, exchange data, and function collectively. This type of testing helps identify issues such as interface mismatches, data transfer errors, and incorrect module interactions. Common approaches to integration testing include top-down, bottom-up, big bang, and incremental methods, each with its own strategy for combining and testing components. The primary goal is to detect integration issues early, ensuring that the overall system performs reliably when all parts are interconnected.

5.2.1 Validation Testing or System Testing

Validation Testing, also known as System Testing, is the final level of software testing where the complete and fully integrated system is evaluated to ensure it meets the specified requirements and performs as expected. It is a black-box testing technique that focuses on validating the behavior, functionality, and performance of the entire system from the user's perspective. During this phase, the application is tested in an environment that closely resembles the production setting, including hardware, software, and network configurations. The objective of validation testing is to verify that the system satisfies both functional and non-functional requirements, such as usability, security, compatibility, and performance. This type of testing helps ensure that the end product is reliable, user-friendly, and ready for deployment, aligning with the business goals and customer expectations.

5.2.2 Output Testing or User Acceptance Testing

Output Testing commonly referred to as User Acceptance Testing (UAT), is the final phase of the software testing lifecycle where the system is evaluated by the end users to ensure it meets their needs and expectations. This testing focuses on verifying that the outputs of the system—such as reports, confirmations, notifications, and visual displays—are accurate, complete, and formatted correctly. During UAT, real-world scenarios and workflows are used to validate that the system behaves as intended under typical usage conditions. It allows stakeholders to confirm that all business requirements have been met and that the system is ready for production. Any discrepancies or issues identified during this stage are addressed before the software goes live. Successful output testing ensures user satisfaction and smooth deployment.

5.2.3 Automation Testing

Automation Testing is a software testing technique that uses specialized tools and scripts to automatically execute test cases, compare actual outcomes with expected results, and report defects. It is primarily used to improve the efficiency, accuracy, and speed of the testing process, especially for repetitive and time-consuming tasks. Automation is highly effective in regression testing, where previously tested functionalities are re-verified after changes or updates in the code. Tools like Selenium, JUnit, TestNG, and others allow testers to simulate user actions and validate system behavior across different browsers, devices, and platforms. By minimizing human intervention, automation testing reduces errors, saves time, and ensures consistent test coverage, making it essential for large-scale, complex, or continuously evolving software applications.

5.2.4 Selenium Testing

Selenium Testing refers to the process of using the Selenium automation tool to test web applications across different browsers and platforms. Selenium is an open-source framework that supports multiple programming languages such as Java, Python, C#, and JavaScript, allowing testers to write test scripts that simulate user interactions with web elements like buttons, forms, links, and navigation flows. It is highly popular due to its flexibility, ease of integration with other tools (like TestNG, JUnit, and Jenkins), and support for cross-browser testing. Selenium consists of several components, including Selenium WebDriver for browser automation, Selenium IDE for record-and-playback testing, and Selenium Grid for parallel test execution. Selenium enables developers and testers to validate web application functionality efficiently, making it a crucial part of modern automated testing strategies.

Example:

Test Case 1: Login Test

Code

```
# Generated by Selenium IDE
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
class TestLogin:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()  # Maximize window to prevent hidden elements
        self.wait = WebDriverWait(self.driver, 10)  # Explicit wait
def teardown_method(self, method):
        self.driver.quit()
def test_login(self):
        self.driver.get("http://localhost/coffeeduplicate/index.php")
 # Wait for login button and click it
        login_button = self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, ".nav-item:nth-child(6) span:nth-child(2)")))
        login_button.click()
# Wait for email field to appear and enter email
        email_field = self.wait.until(EC.visibility_of_element_located((By.NAME, "email")))
        email_field.send_keys("serenitystyles.online@gmail.com")
```

```
# Wait for password field to appear and enter password
password_field = self.wait.until(EC.visibility_of_element_located((By.NAME, "password")))
password_field.send_keys("Serenity@123")
# Click login button
login_btn = self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, ".btn")))
login_btn.click()
# Optional: Wait to confirm login success (Replace with actual success element)
time.sleep(3)
```

Screenshot

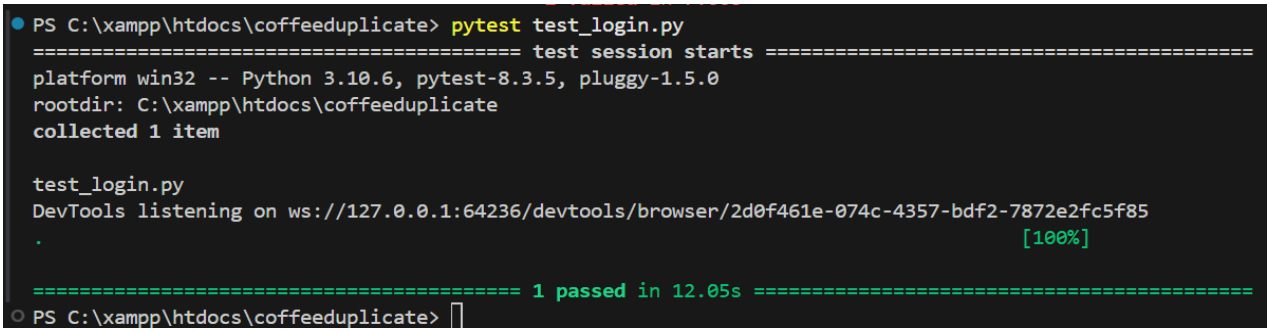


Figure 15 : Login Test successfully

Test Report

Test Case 1

Project Name: Serenity Styles					
Login Test Case					
Test Case ID: Test_1			**Test Designed By: Deepthi C D**		
Test Priority(Low/Medium/High):			**Test Designed Date: 04/04/2025**		
Module Name: Login page			**Test Executed By : Ms.Nimmy Francis**		
Test Title : verify login with valid username and password.			**Test Execution Date: 04/04/2025**		
Description:			User has valid username and password.		
Pre-Condition : User has valid username and password					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/ Fail)**
1	Navigate to the login page		Display login page	Login page displayed	pass
2	Valid username entered	Username:	Admin should enter the main page	Admin entered to the main page	pass
3	Valid password entered	Password:			
4	Click on login button				
Post-Condition: Username and password is checked with database values for successful login					

Test Case 2: Registration test of already existing

Code

```
# Generated by Selenium IDE
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
class TestRegister:
    def setup_method(self, method):
        """Setup WebDriver before each test"""
        options = webdriver.ChromeOptions()
        options.add_argument("--ignore-certificate-errors")  # Ignore SSL issues
        self.driver = webdriver.Chrome(options=options)
        self.driver.implicitly_wait(10)  # Apply implicit wait
        self.vars = {}
    def teardown_method(self, method):
        """Quit the WebDriver after each test"""
        self.driver.quit()
    def test_register(self):
        """Test user registration and OTP verification"""
        self.driver.get("http://localhost/coffeeduplicate/index.php")
        self.driver.set_window_size(1296, 688)
        # Click Register Button
        register_button = WebDriverWait(self.driver, 10).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR,          ".nav-item:nth-
```

```
child(7) span:nth-child(2)"))
        )
        register_button.click()
        # Fill Registration Form
        self.fill_input("name", "Aparna")
        self.fill_input("email", "aparnaprasad2027@mca.ajce.in")
        self.fill_input("phone_no", "8590918598")
        self.fill_input("pass", "Aparna@222")
        self.fill_input("conpass", "Aparna@222")
        # Click Submit
        self.click_button(".btn")
        # Handle Success Alert
        self.click_alert()
        # OTP Verification
        self.fill_input("email", "aparnaprasad2027@mca.ajce.in")
        self.fill_input("otp", "325130")
        self.click_button(".btn")
        self.click_alert()
        print("✅ Registration and OTP test passed successfully!")
    def fill_input(self, name, value):
        """Helper function to input text into a field"""
        field = WebDriverWait(self.driver, 10).until(
            EC.visibility_of_element_located((By.NAME, name))
        )
        self.driver.execute_script("arguments[0].scrollIntoView();", field)  # Ensure
visibility
        field.clear()  # Clear any pre-filled values
```

```
        field.send_keys(value)
    def click_button(self, selector):
        """Helper function to click a button"""
        button = WebDriverWait(self.driver, 10).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, selector))
        )
        self.driver.execute_script("arguments[0].scrollIntoView();", button)  # Ensure visibility
        button.click()
    def click_alert(self):
        """Helper function to handle Swal2 confirmation alerts"""
        try:
            alert = WebDriverWait(self.driver, 10).until(
                EC.element_to_be_clickable((By.CSS_SELECTOR, ".swal2-confirm"))
            )
            alert.click()
        except Exception as e:
            self.driver.save_screenshot("alert_not_found.png")  # Capture screenshot if alert fails
            pytest.fail(f"Failed to click confirmation alert: {e}")
```

Screenshot



Figure 16 : Registration Test successfully

Test Report

Test Case 2

Project Name: Serenity Styles					
User registration Test Case					
Test Case ID: Test_2			**Test Designed By: Deepthi C D**		
Test Priority(Low/Medium/High):			**Test Designed Date: 04/04/2025**		
Module Name: Registration			**Test Executed By : Ms.Nimmy Francis**		
Test Title : registration with available email id and username			**Test Execution Date: 04/04/2025**		
Description: Registration page					
Pre-Condition: User would enter unique email id and username					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/Fail)**
1	Navigate to the registration page		Display registration page	registration page displayed	pass
2	Valid full name entered	Eza Anoop	User should successfully registered and login should be visible	User successfully registered	pass
3	Valid email id entered	eza@gmail.com			
4	Valid phone number entered	8590918598			
5	Valid username entered	eza@gmail.com			
6	Valid password entered	Eza@111			
7	Rentered the same password for confirmation	Eza@111			
8	Click on submit button				
Post-Condition: email id and username should be unique otherwise the email id and username already exist message will display.					

Test Case 3: Add product test

Code

```
# Generated by Selenium IDE
import pytest
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
class TestRegister:
    def setup_method(self, method):
        """Setup WebDriver before each test"""
        options = webdriver.ChromeOptions()
        options.add_argument("--ignore-certificate-errors")  # Ignore SSL issues
        self.driver = webdriver.Chrome(options=options)
        self.driver.implicitly_wait(10)  # Apply implicit wait
        self.vars = {}
    def teardown_method(self, method):
        """Quit the WebDriver after each test"""
        self.driver.quit()
    def test_register(self):
        """Test user registration and OTP verification"""
        self.driver.get("http://localhost/coffeeduplicate/index.php")
        self.driver.set_window_size(1296, 688)
        # Click Register Button
        register_button = WebDriverWait(self.driver, 10).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR,          ".nav-item:nth-
```

```
child(7) span:nth-child(2)"))
        )
        register_button.click()
        # Fill Registration Form
        self.fill_input("name", "Aparna")
        self.fill_input("email", "aparnaprasad2027@mca.ajce.in")
        self.fill_input("phone_no", "8590918598")
        self.fill_input("pass", "Aparna@222")
        self.fill_input("conpass", "Aparna@222")
        # Click Submit
        self.click_button(".btn")
        # Handle Success Alert
        self.click_alert()
        # OTP Verification
        self.fill_input("email", "aparnaprasad2027@mca.ajce.in")
        self.fill_input("otp", "325130")
        self.click_button(".btn")
        self.click_alert()
        print("✅ Registration and OTP test passed successfully!")
    def fill_input(self, name, value):
        """Helper function to input text into a field"""
        field = WebDriverWait(self.driver, 10).until(
            EC.visibility_of_element_located((By.NAME, name))
        )
        self.driver.execute_script("arguments[0].scrollIntoView();", field)  # Ensure
visibility
        field.clear()  # Clear any pre-filled values
```

```
        field.send_keys(value)

    def click_button(self, selector):
        """Helper function to click a button"""
        button = WebDriverWait(self.driver, 10).until(
            EC.element_to_be_clickable((By.CSS_SELECTOR, selector))
        )
        self.driver.execute_script("arguments[0].scrollIntoView();", button)  # Ensure visibility
        button.click()

    def click_alert(self):
        """Helper function to handle Swal2 confirmation alerts"""
        try:
            alert = WebDriverWait(self.driver, 10).until(
                EC.element_to_be_clickable((By.CSS_SELECTOR, ".swal2-confirm"))
            )
            alert.click()
        except Exception as e:
            self.driver.save_screenshot("alert_not_found.png")  # Capture screenshot if alert fails
            pytest.fail(f"Failed to click confirmation alert: {e}")
```

Screenshot

```
test_addproducts.py
DevTools listening on ws://127.0.0.1:53951/devtools/browser/5b30aa7e-b986-4da6-9e2e-763b08d22929
[29992:24428:0405/203137.332:ERROR:ssl_client_socket_impl.cc(877)] handshake failed; returned -1, SSL error code 1, net_error -101
[29992:24428:0405/203137.369:ERROR:ssl_client_socket_impl.cc(877)] handshake failed; returned -1, SSL error code 1, net_error -101
.                                                                                                                    [100%]

============================================== 1 passed in 13.02s ==============================================
PS C:\xampp\htdocs\coffeeduplicate>
```

Figure 17 : Add Product Test successfully

Test Report

Test Case 3

Project Name: Serenity Styles					
Add Product Test Case					
Test Case ID: Test_3			**Test Designed By: Deepthi C D**		
Test Priority(Low/Medium/High):			**Test Designed Date: 04/04/2025**		
Module Name: Add product			**Test Executed By : Ms.Nimmy Francis**		
Test Title : Adding products to the webpage			**Test Execution Date: 04/04/2025**		
Description: Add products page					
Pre-Condition : Admin add products to the webpage					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/ Fail)**
1	Navigate to the add products page		Display add product page	Product page displayed	pass
2	Valid product name entered	Nivea	Admin should successfully add product and should be visible	Admin successfully Added new product	pass
3	Valid category entered	facewash			
4	Valid description entered	Nice product			
5	Valid quantity entered	5			
6	Valid price entered	350			
7	Choose product image	Nivea.png			
8	Click on submit button				
Post-Condition: product name should be unique otherwise the product already exist message will display.					

Test Case 4: Add Service test

Code

```
import pytest
import time
import os
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
class TestAddService:
    def setup_method(self, method):
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 15)
    def teardown_method(self, method):
        self.driver.quit()
    def test_addservice(self):
        driver = self.driver
        wait = self.wait
        # Step 1: Open website
        driver.get("http://localhost/coffeeduplicate/index.php")
        # Step 2: Login
        wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, ".nav-item:nth-child(6) span:nth-child(2)"))).click()
        wait.until(EC.visibility_of_element_located((By.NAME, "email"))).send_keys("serenitystyles.online@gmail.com")
        wait.until(EC.visibility_of_element_located((By.NAME,
```

```
"password"))).send_keys("Serenity@123")
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
".btn"))).click()
    # Step 3: Navigate to Services
    wait.until(EC.element_to_be_clickable((By.LINK_TEXT,
"Services"))).click()
    wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR, "a >
.btn"))).click()
    # Step 4: Fill Service Form
    wait.until(EC.visibility_of_element_located((By.ID,
"service_name"))).send_keys("hair coloring")
    # Step 5: Add New Category
    wait.until(EC.element_to_be_clickable((By.ID, "openModalBtn"))).click()
    wait.until(EC.visibility_of_element_located((By.ID,
"new_category_name"))).send_keys("coloring")
    wait.until(EC.element_to_be_clickable((By.ID, "submit_category"))).click()
    # Wait for modal to disappear (replace ID if different)
    try:
        wait.until(EC.invisibility_of_element_located((By.ID, "categoryModal")))
    except:
        pass
    # Step 6: Fill Remaining Fields
    wait.until(EC.visibility_of_element_located((By.NAME,
"service_description"))).send_keys("give a fantastic look.")
    wait.until(EC.visibility_of_element_located((By.ID,
"service_price"))).send_keys("560")
    # Step 7: Upload Image
    image_path = "C:\\xampp\\htdocs\\coffeeduplicate\\bbluntshampoo1.png"
```

```
assert os.path.exists(image_path), f"Image file not found: {image_path}"

wait.until(EC.presence_of_element_located((By.NAME,
"service_image"))).send_keys(image_path)

# Step 8: Submit Form

submit_btn = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
"button[type='submit']")))

driver.execute_script("arguments[0].scrollIntoView({block: 'center'});",
submit_btn)

time.sleep(1)  # Ensure it's ready

try:

    submit_btn.click()

except:

    driver.execute_script("arguments[0].click();", submit_btn)

# Optional: Wait for success notification/toast

try:

    wait.until(EC.invisibility_of_element_located((By.CLASS_NAME,
"toast")))

except:

    pass
```

Screenshot

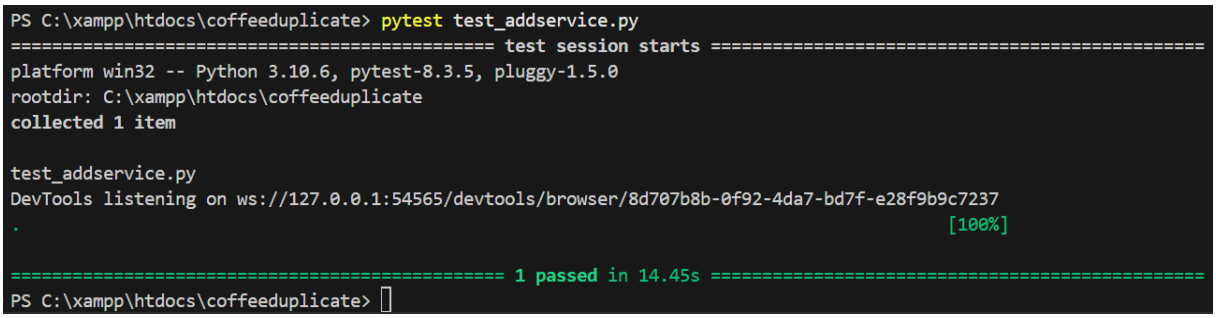


Figure 18 : Add Service Test successfully

Test Report

Test Case 4

Project Name: Serenity Styles					
Add Service Test Case					
Test Case ID: Test_3			**Test Designed By: Deepthi C D**		
Test Priority(Low/Medium/High):			**Test Designed Date: 04/04/2025**		
Module Name: Add Service			**Test Executed By : Ms.Nimmy Francis**		
Test Title : Adding Services to the webpage			**Test Execution Date: 04/04/2025**		
Description: Add Services to webpage					
Pre-Condition : Admin add products to the webpage					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/ Fail)**
1	Navigate to the add service page		Display add service page	Product page displayed	pass
2	Valid service name entered	Bob haircut	Admin should successfully add service and should be visible	Admin successfully Added new service	pass
3	Valid category entered	Haircut			
4	Valid description entered	Nice service			
5	Valid price entered	150			
6	Choose service image	bobhair.png			
8	Click on submit button				
Post-Condition: service name should be unique otherwise the service already exist message will display.					

Test Case 4 : Edit product test

Code

```
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
class TestEditProduct:
    def setup_method(self, method):
        """Setup WebDriver before each test"""
        self.driver = webdriver.Chrome()
        self.driver.maximize_window()
        self.wait = WebDriverWait(self.driver, 10)
    def teardown_method(self, method):
        """Quit WebDriver after each test"""
        self.driver.quit()
    def test_editproduct(self):
        """Test editing an existing product"""
        self.driver.get("http://localhost/coffeeduplicate/index.php")
        # Click login
  login_button=self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
".nav-item:nth-child(6) span:nth-child(2)")))
        login_button.click()
        # Enter email
        email_field = self.wait.until(EC.visibility_of_element_located((By.NAME,
"email")))
        email_field.send_keys("serenitystyles.online@gmail.com")
```

```
        # Enter password
       password_field=self.wait.until(EC.visibility_of_element_located((By.NAME,
"password")))
        password_field.send_keys("Serenity@123")
        # Click login button
       login_btn=self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
".btn")))
        login_btn.click()
        # Navigate to Products
        products_link=self.wait.until(EC.element_to_be_clickable((By.LINK_TEXT,
"Products")))
        products_link.click()
        # Click on Edit (Assumes the first visible edit link — adjust selector if needed)
        edit_link = self.wait.until(EC.element_to_be_clickable((By.LINK_TEXT,
"Edit")))
        edit_link.click()
        # Change product price
        price_input = self.wait.until(EC.visibility_of_element_located((By.NAME,
"price")))
        price_input.clear()
        price_input.send_keys("340")
        # Upload new image (if image is optional, skip or mock it during test)
        image_input = self.wait.until(EC.presence_of_element_located((By.ID,
"product_image")))
        image_path = "C:\\xampp\\htdocs\\coffeeduplicate\\bbluntshampoo1.png" #
Update with a valid image path on your system
        image_input.send_keys(image_path)
```

```
# Submit the form

submit_button=self.wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR
, "button")))

submit_button.click()

# Optional: Assert success message or page change

success_msg = self.wait.until(EC.presence_of_element_located((By.XPATH,
"//div[contains(text(), 'Product updated successfully')]")))

assert "Product updated successfully" in success_msg.text
```

Screenshot

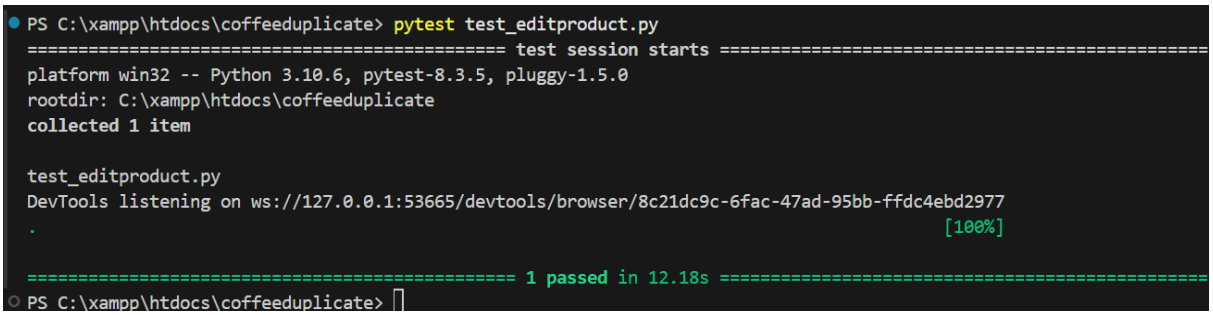


Figure 19 : Add Service Test successfully

Test report

Test Case 5

Project Name: Serenity Styles					
Edit Product Test Case					
Test Case ID: Test_5			**Test Designed By: Deepthi C D**		
Test Priority(Low/Medium/High):			**Test Designed Date: 04/04/2025**		
Module Name: Edit product			**Test Executed By : Ms.Nimmy Francis**		
Test Title : Edit products in the webpage			**Test Execution Date: 04/04/2025**		
Description: Edit product in the webpage					
Pre-Condition : Admin edit products in the webpage					
Step	**Test Step**	**Test Data**	**Expected Result**	**Actual Result**	**Status(Pass/ Fail)**
1	Navigate to the edit product page		Display edit product page	Edit Product page displayed	pass
2	Edit product name entered	plum	Admin should successfully edit product and should be visible	Admin successfully edit the products	pass
3	Edit category entered	Facewash			
4	Edit description entered	Nice product			
5	Edit quantity entered	6			
6	Edit price entered	380			
7	Edit product image	plum.png			
8	Click on submit button				
Post-Condition: Product name should be unique otherwise the product already exist message will display.					

CHAPTER 6

IMPLEMENTATION

6.1 INTRODUCTION

The implementation phase of a project is where the design is transformed into a functional system. It is a crucial stage in ensuring the success of the new system, as it requires gaining user confidence that the system will work effectively and accurately. User training and documentation are key concerns during this phase. Conversion may occur concurrently with user training or at a later stage. Implementation involves the conversion of a newly revised system design into an operational system. During this stage, the user department bears the primary workload, experiences the most significant upheaval, and has the most substantial impact on the existing system. Poorly planned or controlled implementation can cause confusion and chaos. Whether the new system is entirely new, replaces an existing manual or automated system, or modifies an existing system, proper implementation is essential to meet the organization's needs. System implementation involves all activities required to convert from the old to the new system. The system can only be implemented after thorough testing is done and found to be working according to specifications. System personnel evaluate the feasibility of the system. Implementation requires extensive effort in three main areas: education and training, system testing, and changeover. The implementation phase involves careful planning, investigating system and constraints, and designing methods to achieve changeover.

6.2 IMPLEMENTATION PROCEDURES

Software implementation is the process of installing the software in its actual environment and ensuring that it satisfies the intended use and operates as expected. In some organizations, the software development project may be commissioned by someone who will not be using the software themselves. During the initial stages, there may be doubts about the software, but it's important to ensure that resistance does not build up. This can be achieved by:

- Ensuring that active users are aware of the benefits of the new system, building there in the software.
- Providing proper guidance to the users so that they are comfortable using the application. Before viewing the system, users should know that the server program must be running on the server. Without the server object up and running, the intended process will not take place.

6.2.1 User Training

User training is designed to prepare the user for testing and converting the system. To achieve the objective and benefits expected from computer-based system, it is essential for the people

who will be involved to be confident of their role in the new system. As system becomes more complex, the need for training is more important. By user training the user comes to know how to enter data, respond to error messages, interrogate the database and call up routine that will produce reports and perform other necessary functions.

6.2.2 Training on the Application Software

After providing the necessary basic training on computer awareness, it is essential to provide training on the new application software to the user. This training should include the underlying philosophy of using the new system, such as the flow of screens, screen design, the type of help available on the screen, the types of errors that may occur while entering data, and the corresponding validation checks for each entry, and ways to correct the data entered. Additionally, the training should cover information specific to the user or group, which is necessary to use the system or part of the system effectively. It is important to note that this training may differ across different user groups and levels of hierarchy

6.2.3 System Maintenance

The maintenance phase is a crucial aspect of the software development cycle, as it is the time when the software is actually put to use and performs its intended functions. Proper maintenance is essential to ensure that the system remains functional, reliable, and adaptable to changes in the system environment. Maintenance activities go beyond simply identifying and fixing errors or bugs in the system. It may involve updates to the software, modifications to its functionalities, and enhancements to its performance, among other things. In essence, software m aintenance is an ongoing process that requires continuous monitoring, evaluation, and improvement of the system to meet changing user needs and requirements.

6.2.4 Hosting

Hosting is the process of providing a platform or service for storing, serving, and managing websites, applications, or data on the internet. Web hosting refers to the process of making a website accessible on the internet. There are various types of web hosting services available, including shared hosting, VPS hosting, dedicated hosting, cloud hosting, and managed hosting. Shared hosting is a cost-effective option where multiple websites share server resources. VPS hosting offers more control and flexibility over server resources, while dedicated hosting provides exclusive use of a server. Cloud hosting is a scalable and flexible hosting option that utilizes multiple servers, while managed hosting involves the hosting provider taking care of the technical aspects of website management. Cloud hosting offers scalability and high availability by leveraging multiple servers to host websites and applications. Managed hosting involves

outsourcing the technical aspects of hosting to a third-party provider, allowing businesses to focus on their core activities. Choosing the right hosting provider is crucial, as it affects website performance, security, and uptime. Factors to consider when selecting a hosting provider include cost, reliability, support, scalability, and features. Overall, hosting plays a critical role in enabling businesses and individuals to establish a presence on the internet and reach a global audience

AWS Hosting

AWS Hosting is a cloud-based web hosting service provided by Amazon Web Services (AWS), designed for individuals, startups, and large enterprises. AWS offers flexible, scalable, and secure hosting solutions for a wide range of websites and web applications. It is a leading choice for developers and businesses seeking reliable performance and global availability.

Some key features of AWS Hosting include:

- **Scalable infrastructure:**

  Automatically adjusts resources based on website traffic for consistent performance.

- **Flexible hosting options:**

  Supports static (S3), dynamic (EC2), and managed hosting (Elastic Beanstalk, Amplify).

- **,• Global availability:**

  Worldwide data centers ensure low-latency access across regions.

- **Robust security:**

  Includes DDoS protection, SSL, firewalls, and IAM for secure hosting.

- **Developer-friendly:**

  Offers integrations with Git, WordPress, and CI/CD tools, plus full server control.

Procedure for Hosting a Website on AWS:

Step 1: Create an AWS Account:

Go to aws.amazon.com and sign up for an account. You may need to provide billing details and verify your identity.

Step 2: Log in to AWS Management Console:

Once your account is created, log in to the AWS Management Console using your credentials.

Step 3: Choose a Hosting Method:

- Decide how you want to host your website:
- Static website: Use Amazon S3.
- Dynamic website: Use Amazon EC2.
- Managed hosting: Use AWS Elastic Beanstalk or AWS Amplify.

Step 4: Set Up a Hosting Service:

For static sites:

- Go to the S3 service.
- Create a new bucket and enable static website hosting.
- Upload your HTML, CSS, and JavaScript files.

For dynamic sites:

- Launch an EC2 instance and configure a web server (e.g., Apache or Nginx).
- Deploy your application code to the instance.

Step 5: Configure Domain (Optional):

To use a custom domain, set up Amazon Route 53 or configure your domain registrar's DNS to point to your AWS-hosted resources.

Step 6: Set Up Database (If Needed):

Use Amazon RDS or Amazon DynamoDB to host databases for websites requiring dynamic content or user data.

Step 7: Secure Your Site:

Enable HTTPS using AWS Certificate Manager and configure SSL/TLS settings. Use IAM for access control and Security Groups for firewall rules.

Step 8: Test Your Website:

Visit your website using the S3 endpoint, public EC2 IP address, or your custom domain to ensure everything is working properly.

Step 9: Monitor & Optimize:

Use Amazon CloudWatch to track performance, uptime, and errors. Adjust scaling and resources as needed.

Step 10: Regular Maintenance:

Keep your code updated, manage backups, monitor security alerts, and review usage to optimize costs.

Hosted Website : AWS

Hosted Link : http://serenitystyles.teamimpact.in/Serenity-Styles/

Hosted Link QR Code



Screenshot

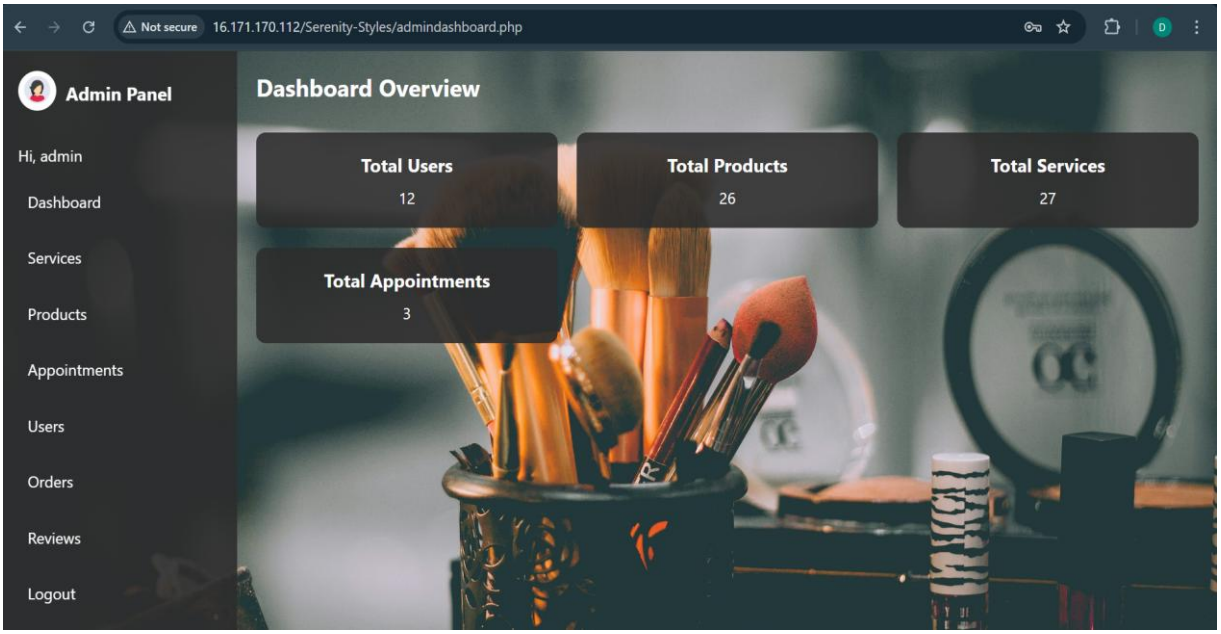


Figure 20: Admin Dashboard

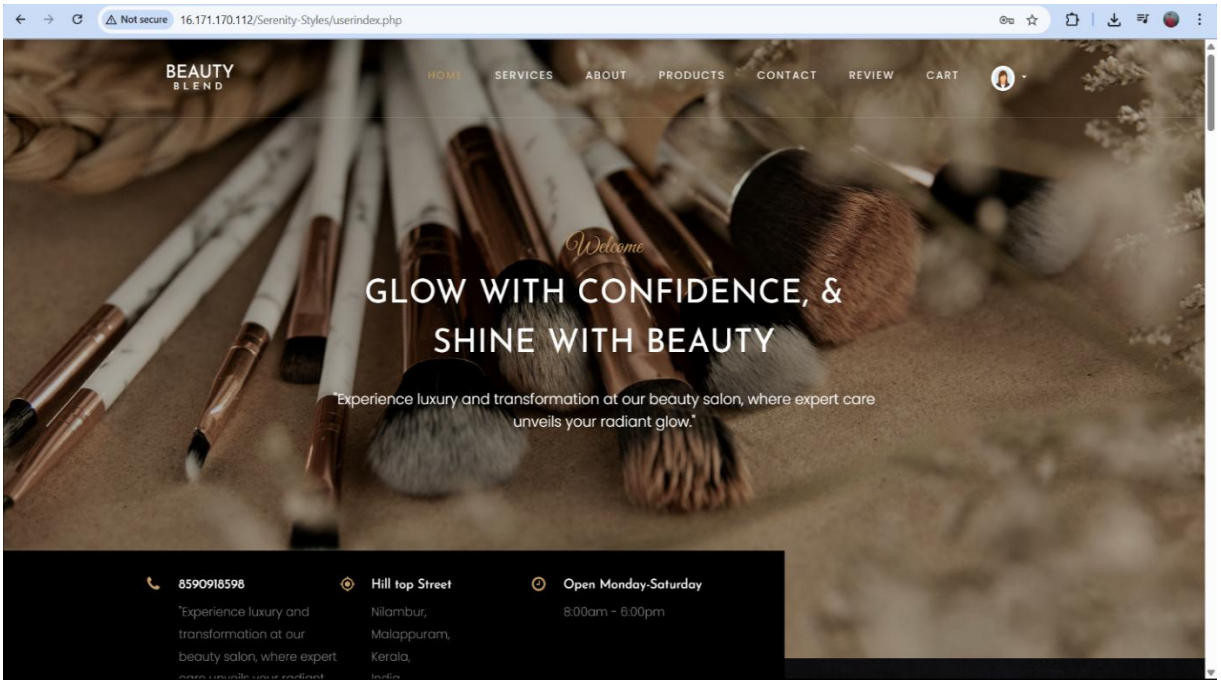


Figure 21 : User Dashboard

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

Serenity Styles has successfully streamlined the appointment scheduling process for beauty salons through an intuitive, real-time web application. By offering features such as dynamic booking, secure payment integration, and service filtering, the platform significantly enhances the convenience and efficiency of salon visits for customers.

The inclusion of a responsive design ensures accessibility across all devices, while the Admin Dashboard empowers salon administrators to effectively manage appointments, services, and user interactions. The integration of customer reviews and ratings further contributes to service quality and user trust.

Looking ahead, Serenity Styles aims to expand its capabilities by incorporating advanced analytics, personalized service recommendations, and broader payment options. Through continuous innovation and user-focused improvements, the platform is poised to maintain its competitive edge in the digital salon management landscape.

7.2 FUTURE SCOPE

Looking ahead, Serenity Styles is poised for strategic growth through innovation, enhanced customer engagement, and service excellence in the beauty and wellness sector. The platform aims to expand its feature set by introducing AI-driven appointment suggestions, personalized service recommendations, and loyalty programs to improve user retention and satisfaction.

To reach a wider audience, Serenity Styles plans to collaborate with a larger network of salons, including niche and premium service providers, across various regions. Global expansion is also in focus, with tailored marketing campaigns and multilingual support to cater to diverse customer bases.

The integration of advanced technologies such as augmented reality (AR) for virtual try-ons and virtual consultations is envisioned to offer immersive pre-service experiences. Additionally, Serenity Styles will continue to invest in its mobile app experience, ensuring a seamless, responsive interface across all devices.

In line with growing sustainability trends, the platform also intends to promote eco-conscious salons and services, empowering customers to make greener beauty choices. Enhanced analytics for salon partners will be introduced to help them understand customer preferences, manage schedules efficiently, and improve service delivery.

By embracing digital transformation, prioritizing customer-centric innovation, and remaining agile to market shifts, Serenity Styles is set to redefine the salon booking experience and strengthen its position as a leader in digital salon management solutions.

CHAPTER 8

BIBLIOGRAPHY

REFERENCES:

- PankajJalote, “Software engineering: a precise approach”
- Gary B. Shelly, Harry J. Rosenblatt, “System Analysis and Design”, 2009
- Ken Schwaber, Mike Beedle, ¯Agile Software Development with ScrumPearson(2008)
- Roger S Pressman, “Software Engineering”

WEBSITES:

- https://billu.co.in/
- https://naturals.in/
- https://stylup.in/
- https://bookinbeautiful.com/en

CHAPTER 9

APPENDIX

9.1 SAMPLE CODE

Add Product

```
<?php
include 'connect.php';
// Security check to prevent unauthorized access
if (!isset($_SESSION['username'])) {
    header('Location: login.php');
    exit();
}
// Add new category via AJAX
if (isset($_POST['ajax_add_category'])) {
    $new_category_name = $_POST['new_category_name'];
    // Insert the new category into the database
    $insert_category_sql = "INSERT INTO tbl_category (category_name, status) VALUES ('$new_category_name', 'available')";
    if ($conn->query($insert_category_sql) === TRUE) {
        $new_id = $conn->insert_id; // Get the ID of the newly inserted category
        // Return the new category data as JSON for the dropdown update
        echo json_encode([
            'success' => true,
            'message' => 'Category added successfully',
            'id' => $new_id,
            'name' => $new_category_name
        ]);
        exit;
    } else {
        echo json_encode(['success' => false, 'message' => 'Error adding category: ' . $conn->error]);
        exit;
    }
}
// Regular form processing for adding categories (non-AJAX fallback)
if (isset($_POST['add_category'])) {
```

```
    $new_category_name = $_POST['new_category_name'];
    // Insert the new category into the database
    $insert_category_sql = "INSERT INTO tbl_category (category_name, status) VALUES
('$new_category_name', 'available')";
    if ($conn->query($insert_category_sql) === TRUE) {
        echo "<div class='alert alert-success'>New category added successfully!</div>";
    } else {
        echo "<div class='alert alert-danger'>Error adding category: " . $conn->error . "</div>";
    }
}
// Fetch categories from the database
$sql = "SELECT category_id, category_name FROM tbl_category WHERE
status='available'";
$result = $conn->query($sql);
// Handle form submission
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["product_name"])) {
    $product_name = $_POST["product_name"];
    $pro_description = $_POST["pro_description"];
    $price = $_POST["price"];
    $quantity = $_POST["quantity"];
    $category = $_POST["category"];

    // Handle file upload
    $target_dir = "images/product_images/"; //location of the image where it stored
    $target_file = $target_dir . basename($_FILES["product_image"]["name"]);
    move_uploaded_file($_FILES["product_image"]["tmp_name"], $target_file); //move
uploaded file to the directory
    $sql = "INSERT INTO tbl_products (product_name, pro_description, price, quantity,
category_id, product_image)
            VALUES ('$product_name', '$pro_description', '$price', '$quantity', '$category',
'$target_file')";
    if ($conn->query($sql) === TRUE) {
        echo "<div class='alert alert-success'>New product added successfully, redirecting to
product list</div>";
```

```
        // Wait for 3 seconds and redirect
        echo "<script>
                setTimeout(function() {
                    window.location.href = 'viewproducts.php';
                }, 3000);
              </script>";
    } else {
        echo "<div class='alert alert-danger'>Error: " . $conn->error . "</div>";
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Dashboard</title>
    <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
    <style>
        body {
            background-color: #1a1a1a;
            color: #fff;
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        }
        .sidebar {
            width: 250px;
            background-color: #2d2d2d;
            height: 100vh;
            padding: 20px;
            position: fixed;
        }
        .sidebar-header {
```

```
    margin-bottom: 30px;
    color: #fff;
}
.nav-links {
    list-style: none;
    padding: 0;
}
.nav-links li {
    margin-bottom: 15px;
}
.nav-links a {
    color: #fff;
    text-decoration: none;
    padding: 10px;
    display: block;
    border-radius: 5px;
    transition: background-color 0.3s;
}
.nav-links a:hover {
    background-color: #3d3d3d;
}
.main-content {
    margin-left: 270px;
    padding: 20px;
}
.table-container,
.form-container {
    background-color: #2d2d2d;
    padding: 20px;
    border-radius: 10px;
    margin-bottom: 30px;
}
```

```
table {
    width: 100%;
    border-collapse: collapse;
    color: #fff;
}
th,
td {
    padding: 12px;
    text-align: left;
    border-bottom: 1px solid #3d3d3d;
}
th {
    background-color: #4a4a4a;
}
.form-container input,
.form-container textarea,
.form-container select {
    width: 100%;
    padding: 10px;
    margin-bottom: 10px;
    border-radius: 5px;
    border: 1px solid #3d3d3d;
    background-color: #1a1a1a;
    color: #fff;
}
.form-container button {
    padding: 10px 20px;
    background-color: #4CAF50;
    border: none;
    color: #fff;
    border-radius: 5px;
    cursor: pointer;
```

```
    transition: background-color 0.3s;

.form-container button:hover {
    background-color: #45a049;
}
/* Category button styles */
.category-actions {
    display: flex;
    align-items: center;
    margin-bottom: 10px;
}
.add-category-btn {
    background-color: #007bff;
    color: white;
    border: none;
    padding: 5px 10px;
    border-radius: 5px;
    margin-left: 10px;
    cursor: pointer;
}
.add-category-btn:hover {
    background-color: #0069d9;
}
/* Modal styles */
.modal {
    display: none;
    position: fixed;
    z-index: 1;
    left: 0;
    top: 0;
    width: 100%;
    height: 100%;
```

```
            overflow: auto;
            background-color: rgba(0, 0, 0, 0.4);
        }
        .modal-content {
            background-color: #2d2d2d;
            margin: 15% auto;
            padding: 20px;
            border: 1px solid #3d3d3d;
            border-radius: 5px;
            width: 50%;
            color: #fff;
        }
        .close {
            color: #aaa;
            float: right;
            font-size: 28px;
            font-weight: bold;
        }
        .close:hover,
        .close:focus {
            color: #fff;
            text-decoration: none;
            cursor: pointer;
        }
        .alert {
            margin-top: 10px;
            margin-bottom: 10px;
        }
    </style>
</head>
<body>
    <div class="sidebar">
```

```
    <div class="sidebar-header">
      <h3>Admin Panel</h3>
      <p><?php echo "Hi, " . $_SESSION['username']; ?></p>
    </div>
    <ul class="nav-links">
      <li><a href="admindashboard.php">Dashboard</a></li>
      <li><a href="viewservices.php">Services</a></li>
      <li><a href="viewproducts.php">Products</a></li>
      <li><a href="viewappointments.php">Appointments</a></li>
      <li><a href="viewuser.php">Users</a></li>
      <li><a href="logout.php">Logout</a></li>
    </ul>
  </div>
  <div class="main-content">
    <h2>Manage Products</h2>

    <!-- Status message area for feedback -->
    <div id="status-message"></div>
    <!-- Add New Product Form -->
    <div class="form-container">
      <h5>Add New Product</h5>
      <form method="post" id="product_form" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]); ?>"
        enctype="multipart/form-data">
        <input type="text" name="product_name" id="product_name"
placeholder="Product Name" required>
        <!-- Category selection with Add New button -->
        <div class="category-actions">
          <select name="category" id="category" required style="width: calc(100% -
120px);">
            <option value="" disabled selected>Select Category</option>
            <?php
            if ($result->num_rows > 0) {
```

```
                    // Output data for each row
                    while ($row = $result->fetch_assoc()) {
                        echo '<option value="' . htmlspecialchars($row['category_id']) . '">' .
htmlspecialchars($row['category_name']) . '</option>';
                    }
                } else {
                    echo '<option value="" disabled>No categories available</option>';
                }
                ?>
            </select>
            <button type="button" class="add-category-btn" id="openModalBtn">Add New
Category</button>
        </div>
        <textarea name="pro_description" placeholder="Description" rows="4"
required></textarea>
        <input type="number" name="quantity" placeholder="Quantity" required>
        <input type="number" name="price" id="product_price" placeholder="Price"
step="0.01" required>
        <div class="preview">
            <!-- Image preview -->
            <img id="image_preview" src="#" alt="Image Preview"
                style="display: none; max-width: 200px; margin-top: 10px;">
            <!-- File input for product image -->
            <input type="file" name="product_image" accept="image/*" required
onchange="previewImage(event)">
        </div>
        <div>
            <button type="submit">Add Product</button>
        </div>
    </form>
  </div>
</div>
<!-- Add Category Modal -->
<div id="categoryModal" class="modal">
```

```
        <div class="modal-content">
            <span class="close">&times;</span>
            <h4>Add New Category</h4>
            <form method="post" id="category_form">
                <input type="text" name="new_category_name" id="new_category_name"
placeholder="Category Name" required>
                <button type="submit" id="submit_category">Add Category</button>
            </form>
            <div id="modal-status-message"></div>
        </div>
    </div>
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/jquery-
validation@1.19.5/dist/jquery.validate.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
        crossorigin="anonymous"></script>
    <script>
        // Image preview function
        function previewImage(event) {
            var reader = new FileReader();
            reader.onload = function () {
                var output = document.getElementById('image_preview');
                output.src = reader.result;
                output.style.display = 'block';
            }
            reader.readAsDataURL(event.target.files[0]);
        }
        // Modal functionality
        var modal = document.getElementById("categoryModal");
        var btn = document.getElementById("openModalBtn");
        var span = document.getElementsByClassName("close")[0];
```

```
btn.onclick = function () {
    modal.style.display = "block";
}
span.onclick = function () {
    modal.style.display = "none";
}
window.onclick = function (event) {
    if (event.target == modal) {
        modal.style.display = "none";
    }
}
$(document).ready(function () {
    // Add category form submission with AJAX
    $("#category_form").submit(function (e) {
        e.preventDefault();

        var categoryName = $("#new_category_name").val();

        if (categoryName.trim() === '') {
            $("#modal-status-message").html('<div class="alert alert-danger">Category name cannot be empty</div>');
            return false;
        }
        $.ajax({
            type: "POST",
            url: "<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>",
            data: {
                ajax_add_category: true,
                new_category_name: categoryName
            },
            dataType: "json",
            success: function (response) {
                if (response.success) {
```

```
                    // Add the new category to the dropdown directly
                    var newOption = new Option(response.name, response.id);
                    $("#category").append(newOption);

                    // Select the newly added category
                    $("#category").val(response.id);

                    // Show success message
                    $("#modal-status-message").html('<div class="alert alert-success">' +
response.message + '</div>');
                    // Clear the form
                    $("#new_category_name").val("");
                    // Close the modal after 1.5 seconds
                    setTimeout(function () {
                        modal.style.display = "none";
                        $("#modal-status-message").html('');
                    }, 1500);
                } else {
                    // Show error message
                    $("#modal-status-message").html('<div class="alert alert-danger">' +
response.message + '</div>');
                }
            },
            error: function () {
                $("#modal-status-message").html('<div class="alert alert-danger">Error adding
category. Please try again.</div>');
            }
        });
    });
    // Custom validation methods
    jQuery.validator.addMethod('lettersonly', function (value, element) {
        return /^[^-\s][a-zA-Z_\s-]+$/.test(value);
    }, "Please use letters only.");
```

```
        jQuery.validator.addMethod('valid_Quantity', function (value, element) {
            return /^[+]?\d+$/.test(value) && parseInt(value, 10) > 0;
        }, "Quantity should be a positive whole number greater than 0");
        jQuery.validator.addMethod('valid_Productprice', function (value, element) {
            return parseFloat(value) >= 100;
        }, "Price should be minimum 100");
        jQuery.validator.addMethod('valid_Productmaxprice', function (value, element) {
            return parseFloat(value) < 10000;
        }, "Price should be maximum 10000");
        jQuery.validator.addMethod('productImage', function (value, element, param) {
            var extension = value.substring(value.lastIndexOf('.') + 1).toLowerCase();
            return extension === 'png' || extension === 'jpg' || extension === 'jpeg' || extension
=== 'svg';
        }, "Please select a valid image file (PNG, JPG, JPEG, or SVG).");
        // Form validation rules and messages
        $('#product_form').validate({
            rules: {
                product_name: {
                    required: true,
                    lettersonly: true,
                    minlength: 3,
                    remote: {
                        url: "check_product.php",
                        type: "POST",
                        data: {
                            username: function () {
                                return $("#product_name").val();
                            }
                        }
                    }
                },
                category: {
                    required: true
```

```
        },
        pro_description: {
            required: true
        },
        quantity: {
            required: true,
            valid_Quantity: true
        },
        price: {
            required: true,
            number: true,
            valid_Productprice: true,
            valid_Productmaxprice: true
        },
        product_image: {
            required: true,
            productImage: true
        }
    },
    messages: {
        product_name: {
            required: "Please enter your full name",
            lettersonly: "Name must be in alphabets only",
            remote: "Product already exists!"
        },
        category: {
            required: "Please select a category"
        },
        pro_description: {
            required: "Product description is required"
        },
        quantity: {
```

```
            required: "Please enter a quantity",
            valid_Quantity: "Quantity must be a whole number greater than 0"
        },
        price: {
            required: "Price must be entered",
            number: "Enter a valid price",
            valid_Productprice: "Price should be minimum 100",
            valid_Productmaxprice: "Price should be maximum 10000"
        },
        product_image: {
            required: "Please upload product image",
            productImage: "Please select a valid image file (PNG, JPG, JPEG, or SVG)."
        }
    },
    errorPlacement: function (error, element) {
        error.insertAfter(element);
    }
});
// Add validation for category form
$('#category_form').validate({
    rules: {
        new_category_name: {
            required: true,
            lettersonly: true,
            minlength: 3
        }
    },
    messages: {
        new_category_name: {
            required: "Please enter a category name",
            lettersonly: "Category name must be alphabets only",
            minlength: "Category name must be at least 3 characters"
```

```
            }
        }
    });
    $('#product_price').change(function () {
        $('#product_price').valid();
    });
});
</script>
</body>
</html>
```

Add Services

```
<?php
include 'connect.php';
// Security check to prevent unauthorized access
if (!isset($_SESSION['username'])) {
    header('Location: login.php');
    exit();
}
// Add new category via AJAX
if (isset($_POST['ajax_add_category'])) {
    $new_category_name = $_POST['new_category_name'];

    // Insert the new category into the database
    $insert_category_sql = "INSERT INTO tbl_category_services (cat_name, status) VALUES ('$new_category_name', 'active')";
    if ($conn->query($insert_category_sql) === TRUE) {
        $new_id = $conn->insert_id; // Get the ID of the newly inserted category
        // Return the new category data as JSON for the dropdown update
        echo json_encode([
            'success' => true,
            'message' => 'Category added successfully',
            'id' => $new_id,
            'name' => $new_category_name
```

```
        ]);
        exit;
    } else {
        echo json_encode(['success' => false, 'message' => 'Error adding category: ' . $conn->error]);
        exit;
    }
}
// Regular form processing for adding categories (non-AJAX fallback)
if (isset($_POST['add_category'])) {
    $new_category_name = $_POST['new_category_name'];
        // Insert the new category into the database
    $insert_category_sql = "INSERT INTO tbl_category_services (cat_name, status) VALUES ('$new_category_name', 'active')";
        if ($conn->query($insert_category_sql) === TRUE) {
        echo "<div class='alert alert-success'>New category added successfully!</div>";
    } else {
        echo "<div class='alert alert-danger'>Error adding category: " . $conn->error . "</div>";
    }}
// Fetch categories from the database
$sql = "SELECT catservice_id, cat_name FROM tbl_category_services WHERE status='active'";result = $conn->query($sql);
// Handle form submission for adding a service
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST["service_name"])) {
    $service_name = $_POST["service_name"] ?? '';
    $service_description = $_POST["service_description"] ?? '';
    $category_service = $_POST["category_service"] ?? '';
    $service_price = $_POST["service_price"] ?? '';

    // Handle file upload
    $target_dir = "images/service_images/"; //location of the image where it stored
    $target_file = $target_dir . basename($_FILES["service_image"]["name"]);
    move_uploaded_file($_FILES["service_image"]["tmp_name"], $target_file); //move uploaded file to the directory
```

```
    // Insert into database only if image upload is successful or not required
  $sql = "INSERT INTO tbl_services (service_name, service_description, catservice_id, price, service_image)
          VALUES ('$service_name', '$service_description', '$category_service', '$service_price', '$target_file')";
  if ($conn->query($sql) === TRUE) {
    echo "<div class='alert alert-success'>New service added successfully, redirecting to service list</div>";
    echo "<script>setTimeout(function() { window.location.href = 'viewservices.php'; }, 3000);</script>";
  } else {
    echo "<div class='alert alert-danger'>Error: " . $conn->error . "</div>";
  }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Dashboard</title>
  <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css">
  <style>
    body {
      background-color: #1a1a1a;
      color: #fff;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    }
    .sidebar {
      width: 250px;
      background-color: #2d2d2d;
      height: 100vh;
      padding: 20px;
```

```
    position: fixed;
}
.sidebar-header {
    margin-bottom: 30px;
    color: #fff;
}
.nav-links {
    list-style: none;
    padding: 0;
}
.nav-links li {
    margin-bottom: 15px;
}
.nav-links a {
    color: #fff;
    text-decoration: none;
    padding: 10px;
    display: block;
    border-radius: 5px;
    transition: background-color 0.3s;
}
.nav-links a:hover {
    background-color: #3d3d3d;
}
.main-content {
    margin-left: 270px;
    padding: 20px;
}

.table-container,
.form-container {
    background-color: #2d2d2d;
```

```
    padding: 20px;
    border-radius: 10px;
    margin-bottom: 30px;
}
table {
    width: 100%;
    border-collapse: collapse;
    color: #fff;
}
th,
td {
    padding: 12px;
    text-align: left;
    border-bottom: 1px solid #3d3d3d;
}
th {
    background-color: #4a4a4a;
}
.form-container input,
.form-container textarea,
.form-container select {
    width: 100%;
    padding: 10px;
    margin-bottom: 10px;
    border-radius: 5px;
    border: 1px solid #3d3d3d;
    background-color: #1a1a1a;
    color: #fff;
}
.form-container button {
    padding: 10px 20px;
    background-color: #4CAF50;
```

```
    border: none;
    color: #fff;
    border-radius: 5px;
    cursor: pointer;
    transition: background-color 0.3s;
}
.form-container button:hover {
    background-color: #45a049;
}
        /* Category button styles */
.category-actions {
    display: flex;
    align-items: center;
    margin-bottom: 10px;
}

.add-category-btn {
    background-color: #007bff;
    color: white;
    border: none;
    padding: 5px 10px;
    border-radius: 5px;
    margin-left: 10px;
    cursor: pointer;
}
      .add-category-btn:hover {
    background-color: #0069d9;
           /* Modal styles */
.modal {
    display: none;
    position: fixed;
    z-index: 1;
```

```
    left: 0;
    top: 0;
    width: 100%;
    height: 100%;
    overflow: auto;
    background-color: rgba(0,0,0,0.4);
}
      .modal-content {
    background-color: #2d2d2d;
    margin: 15% auto;
    padding: 20px;
    border: 1px solid #3d3d3d;
    border-radius: 5px;
    width: 50%;
    color: #fff;
}
.close {
    color: #aaa;
    float: right;
    font-size: 28px;
    font-weight: bold;
}
.close:hover,
.close:focus {
    color: #fff;
    text-decoration: none;
    cursor: pointer;
}
.alert {
    margin-top: 10px;
    margin-bottom: 10px;
}
```

```
    </style>
</head>
<body>
    <div class="sidebar">
        <div class="sidebar-header">
            <h3>Admin Panel</h3>
            <p><?php echo "Hi, " . $_SESSION['username']; ?></p>
        </div>
        <ul class="nav-links">
            <li><a href="admindashboard.php">Dashboard</a></li>
            <li><a href="addservices.php">Services</a></li>
            <li><a href="addproducts.php">Products</a></li>
            <li><a href="viewappointments.php">Appointments</a></li>
            <li><a href="viewuser.php">Users</a></li>
            <li><a href="logout.php">Logout</a></li>
        </ul>
    </div>
    <div class="main-content">
        <h2>Manage Services</h2>
        <!-- Status message area for feedback -->
        <div id="status-message"></div>
        <!-- Add New Service Form -->
        <div class="form-container">
            <h5>Add New Service</h5>
            <form method="post" id="service_form" action="<?php echo
htmlspecialchars($_SERVER["PHP_SELF"]); ?>" enctype="multipart/form-data">
                <input type="text" name="service_name" id="service_name" placeholder="Service
Name" required>
                        <!-- Category selection with Add New button -->
                <div class="category-actions">
                    <select name="category_service" id="category_service" required style="width:
calc(100% - 120px);">
                        <option value="" disabled selected>Select Category</option>
```

```
                <?php
                if ($result->num_rows > 0) {
                    // Output data for each row
                    while ($row = $result->fetch_assoc()) {
                        echo '<option value="' . htmlspecialchars($row['catservice_id']) . '">' .
htmlspecialchars($row['cat_name']) . '</option>';
                    }
                } else {
                    echo '<option value="" disabled>No categories available</option>';
                }
                ?>
            </select>
            <button type="button" class="add-category-btn" id="openModalBtn">Add New
Category</button>
        </div>
                    <textarea name="service_description" placeholder="Description"
rows="4" required></textarea>
        <input type="number" name="service_price" id="service_price"
placeholder="Price" required>
        <div class="preview">
            <!-- Image preview -->
            <img id="image_preview" src="#" alt="Image Preview" style="display: none;
max-width: 200px; margin-top: 10px;">
            <!-- File input for product image -->
            <input type="file" name="service_image" accept="image/*" required
onchange="previewImage(event)">
        </div>
        <div>
            <button type="submit">Add Service</button>
        </div>
      </form>
    </div>
  </div>
    <!-- Add Category Modal -->
```

```
    <div id="categoryModal" class="modal">
        <div class="modal-content">
            <span class="close">&times;</span>
            <h4>Add New Category</h4>
            <form method="post" id="category_form">
                <input type="text" name="new_category_name" id="new_category_name"
placeholder="Category Name" required>
                <button type="submit" id="submit_category">Add Category</button>
            </form>
            <div id="modal-status-message"></div>
        </div>
    </div>
        <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/jquery-
validation@1.19.5/dist/jquery.validate.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
crossorigin="anonymous"></script>
        <script>
    // Image preview function
    function previewImage(event) {
        var reader = new FileReader();
        reader.onload = function() {
            var output = document.getElementById('image_preview');
            output.src = reader.result;
            output.style.display = 'block';
        }
        reader.readAsDataURL(event.target.files[0]);
    }
        // Modal functionality
    var modal = document.getElementById("categoryModal");
    var btn = document.getElementById("openModalBtn");
    var span = document.getElementsByClassName("close")[0];
```

```
    btn.onclick = function() {
    modal.style.display = "block";
  }
  span.onclick = function() {
    modal.style.display = "none";
  }
  window.onclick = function(event) {
    if (event.target == modal) {
      modal.style.display = "none";
    }
  }
    $(document).ready(function () {
    // Add category form submission with AJAX
    $("#category_form").submit(function(e) {
      e.preventDefault();
      var categoryName = $("#new_category_name").val();
            if (categoryName.trim() === '') {
        $("#modal-status-message").html('<div class="alert alert-danger">Category name cannot be empty</div>');
        return false;
      }
            $.ajax({
        type: "POST",
        url: "<?php echo htmlspecialchars($_SERVER["PHP_SELF"]); ?>",
        data: {
          ajax_add_category: true,
          new_category_name: categoryName
        },
        dataType: "json",
        success: function(response) {
          if (response.success) {
            // Add the new category to the dropdown directly
            var newOption = new Option(response.name, response.id);
```

```
                $("#category_service").append(newOption);
                // Select the newly added category
                $("#category_service").val(response.id);
                // Show success message
                $("#modal-status-message").html('<div class="alert alert-success">' +
response.message + '</div>');
                // Clear the form
                $("#new_category_name").val("");
                // Close the modal after 1.5 seconds
                setTimeout(function() {
                    modal.style.display = "none";
                    $("#modal-status-message").html('');
                }, 1500);
            } else {
                // Show error message
                $("#modal-status-message").html('<div class="alert alert-danger">' +
response.message + '</div>');
            }
        },
        error: function() {
            $("#modal-status-message").html('<div class="alert alert-danger">Error adding
category. Please try again.</div>');
        }
    });
});
        // Custom validation methods
    jQuery.validator.addMethod('lettersonly', function (value, element) {
        return /^[^-\s][a-zA-Z_\s-]+$/.test(value);
    }, "Please use letters only.");
    jQuery.validator.addMethod('valid_Price', function (value, element) {
        return /^[+]?(?:\d+|\d{1,3}(?:,\d{3})+)(?:\.\d+)?$/.test(value) && parseFloat(value) >=
100;
    }, "Price should be minimum 100");
    jQuery.validator.addMethod('serviceImage', function (value, element, param) {
```

```
        var extension = value.substring(value.lastIndexOf('.') + 1).toLowerCase();
        return extension === 'png' || extension === 'jpg' || extension === 'jpeg' || extension === 'svg';
    }, "Please select a valid image file (PNG, JPG, JPEG, or SVG).");
    jQuery.validator.addMethod('maxPricePerCategory', function (value, element) {
        var category = $('#category_service').val();
        var price = parseFloat(value);
        var maxPrices = {
            '1': 1200,
            '2': 20000,
            '3': 5000,
            '4': 50000
        };
        return price <= (maxPrices[category] || Infinity); // Default to no limit if category not found
    }, "Price exceeds the maximum allowed for the selected category.");
    // Form validation rules and messages
    $('#service_form').validate({
        rules: {
            service_name: {
                required: true,
                lettersonly: true,
                minlength: 3,
                remote: {
                    url: "check_service.php",
                    type: "POST",
                    data: {
                        username: function() {
                            return $("#service_name").val();
                        }
                    }
                }
            },
```

```
            category_service: {
                required: true
            },
            service_description: {
                required: true
            },
            service_price: {
                required: true,
                valid_Price: true,
                maxPricePerCategory: true
            },
            service_image: {
                required: true,
                serviceImage: true
            }
        },
        messages: {
            service_name: {
                required: "Please enter your full name",
                lettersonly: "Name must be in alphabets only",
                remote: "Service already exist!"
            },
            category_service: {
                required: "Please select a category"
            },
            service_description: {
                required: "Service description is required"
            },
            service_price: {
                required: "Price must be entered",
                valid_Price: "Price should be minimum 100",
                maxPricePerCategory: "Price exceeds the maximum allowed for the selected
category."
```

```
            },
            service_image: {
                required: "Please upload service image",
                serviceImage: "Please select a valid image file (PNG, JPG, JPEG, or SVG)."
            }
        },
        errorPlacement: function (error, element) {
            error.insertAfter(element);
        }
    });
    // Revalidate price field when category changes
    $('#category_service').change(function () {
        $('#service_price').valid();
    });
    // Add validation for category form
    $('#category_form').validate({
        rules: {
            new_category_name: {
                required: true,
                lettersonly: true,
                minlength: 3
            }
        },
        messages: {
            new_category_name: {
                required: "Please enter a category name",
                lettersonly: "Category name must be alphabets only",
                minlength: "Category name must be at least 3 characters"
            }
        }
    });
});
```

```
    </script>
</body>
</html>
```

Book Appointment

```
<?php
include("connect.php");
if (!empty($_SESSION["email"])) {
    $email = $_SESSION["email"];
    $stmt = $conn->prepare("SELECT u.name FROM tbl_user AS u JOIN tbl_login AS l ON u.user_id = l.user_id WHERE l.email = ?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $result = $stmt->get_result();
    if ($result->num_rows > 0) {
        $row = $result->fetch_assoc();
    }
    $stmt->close();
}
/ Sanitize GET parameters
$user_id = isset($_GET['user_id']) ? intval($_GET['user_id']) : null;
$service_id = isset($_GET['service_id']) ? intval($_GET['service_id']) : null;
$alertMessage = ""; // To store JavaScript alerts
if(!empty($service_id)){
  $sql="SELECT price FROM tbl_services WHERE service_id=$service_id";
  $result=$conn->query($sql);
  if($result->num_rows>0){
    $row=$result->fetch_assoc();
  }
}
// Function to get booked times for a specific date
function getBookedAppointmentTimes($conn, $date) {
  $bookedTimes = array();
  $stmt = $conn->prepare("SELECT appointment_time FROM tbl_appointments WHERE
```

```
appointment_date = ? AND status != 'Cancelled'");
  $stmt->bind_param("s", $date);
  $stmt->execute();
  $result = $stmt->get_result();
    while($row = $result->fetch_assoc()) {
    $bookedTimes[] = $row['appointment_time'];
  }
  $stmt->close();
  return $bookedTimes;
}
?
<!DOCTYPE html>
<html lang="en">
<head>
  <title></title>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
  <link href="https://fonts.googleapis.com/css?family=Poppins:300,400,500,600,700"
rel="stylesheet">
  <link href="https://fonts.googleapis.com/css?family=Josefin+Sans:400,700"
rel="stylesheet">
  <link href="https://fonts.googleapis.com/css?family=Great+Vibes" rel="stylesheet">
  <link rel="stylesheet" href="css/open-iconic-bootstrap.min.css">
  <link rel="stylesheet" href="css/animate.css">
  <link rel="stylesheet" href="css/owl.carousel.min.css">
  <link rel="stylesheet" href="css/owl.theme.default.min.css">
  <link rel="stylesheet" href="css/magnific-popup.css">
  <link rel="stylesheet" href="css/aos.css">
  <link rel="stylesheet" href="css/ionicons.min.css">
  <link rel="stylesheet" href="css/bootstrap-datepicker.css">
  <link rel="stylesheet" href="css/jquery.timepicker.css">


  <link rel="stylesheet" href="css/flaticon.css">
```

```
  <link rel="stylesheet" href="css/icomoon.css">
  <link rel="stylesheet" href="css/style.css">
  <!-- SweetAlert2 -->
  <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark ftco_navbar bg-dark ftco-navbar-light" id="ftco-navbar">
    <div class="container">
      <a class="navbar-brand" href="userindex.php">Beauty<small>Blend</small></a>
      <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#ftco-nav"
        aria-controls="ftco-nav" aria-expanded="false" aria-label="Toggle navigation">
        <span class="oi oi-menu"></span> Menu
      </button>
      <div class="collapse navbar-collapse" id="ftco-nav">
        <ul class="navbar-nav ml-auto">
          <li class="nav-item"><a href="userindex.php" class="nav-link">Home</a></li>
          <li class="nav-item"><a href="services.php" class="nav-link">Services</a></li>
          <li class="nav-item"><a href="about.php" class="nav-link">About</a></li>
          <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="shop.php" id="dropdown04" data-toggle="dropdown"
              aria-haspopup="true" aria-expanded="false">Products</a>
            <div class="dropdown-menu" aria-labelledby="dropdown04">
              <a class="dropdown-item" href="shop.php">Products</a>
              <a class="dropdown-item" href="product-single.php">Single Product</a>
              <a class="dropdown-item" href="cart.php">Cart</a>
              <a class="dropdown-item" href="checkout.php">Checkout</a>
            </div>
          </li>
          <li class="nav-item"><a href="contact.php" class="nav-link">Contact</a></li>
          <li class="nav-item active"><a href="booknow.php" class="nav-link">Book Now</a></li>
```

```
<li class="nav-item"><a href="review.php" class="nav-link">Reviews</a></li>
<?php if (isset($_SESSION['username']) && $_SESSION['username'] === 'user') { ?>
  <li class="nav-item dropdown">
    <a class="nav-link dropdown-toggle" href="#" id="profileDropdown" data-toggle="dropdown"
      aria-haspopup="true" aria-expanded="false">
      <img src="images/profile2.jpg" alt="Profile" id="profile-icon" class="rounded-circle"
        style="width: 30px; height: 30px;">
    </a>
    <?php
    $user_id = $_SESSION['user_id'];
    ?>
    <div class="dropdown-menu" aria-labelledby="profileDropdown">
      <a class="dropdown-item"
        href="profile.php?user_id=<?php echo $user_id; ?>"><?php echo $row['name']; ?></a>
      <a class="dropdown-item" href="profile.php?user_id=<?php echo $user_id; ?>">Profile</a>

      <a class="dropdown-item" href="viewappointments.html?user_id=<?php echo $user_id; ?>">View
        Appointments</a>
      <a class="dropdown-item" href="vieworders.html?user_id=<?php echo $user_id; ?>">View
        Orders</a>
      <a class="dropdown-item" href="logout.php?user_id=<?php echo $user_id; ?>">Log Out</a>
    </div>
  </li>
<?php }
?>
<?php if (isset($_SESSION['username']) && $_SESSION['username'] === 'user') { ?>
```

```
          <li class="nav-item cart"><a href="cart.php" class="nav-link"><span
              class="icon icon-shopping_cart"></span><span
              class="bag d-flex justify-content-center align-items-
center"><small>1</small></span></a>
          </li>
         <?php }
         ?>
        </ul>
      </div>
    </div>
  </nav>
  <!-- END nav -->
  <section class="home-slider owl-carousel">

    <div class="slider-item" style="background-image: url(images/coverpage1.jpg);" data-
stellar-background-ratio="0.5">
      <div class="overlay"></div>
      <div class="container">
        <div class="row slider-text justify-content-center align-items-center">
          <div class="col-md-7 col-sm-12 text-center ftco-animate">
            <h1 class="mb-3 mt-5 bread">BOOK APPOINTMENT</h1>
            <p class="breadcrumbs"><span class="mr-2"><a
href="userindex.php">Home</a></span> <span>Book Now</span></p>
          </div>
        </div>
      </div>
    </div>
  </section>
  <section class="ftco-intro">
    <div class="container-wrap">
      <div class="wrap d-md-flex align-items-xl-end">
        <div class="info">
          <div class="row no-gutters">
```

```
          <div class="col-md-4 d-flex ftco-animate">
            <div class="icon"><span class="icon-phone"></span></div>
            <div class="text">
              <h3>8590918598</h3>
              <p>"Experience luxury and transformation at our beauty salon, where expert care
unveils your radiant
                glow."</p>
            </div>
          </div>
          <div class="col-md-4 d-flex ftco-animate">
            <div class="icon"><span class="icon-my_location"></span></div>
            <div class="text">
              <h3>Hill top Street</h3>
              <p> Nilambur,<br>Malappuram,<br>Kerala,<br>India</p>
            </div>
          </div>
          <div class="col-md-4 d-flex ftco-animate">
            <div class="icon"><span class="icon-clock-o"></span></div>
            <div class="text">
              <h3>Open Monday-Saturday</h3>
              <p>8:00am - 6:00pm</p>
            </div>
          </div>
        </div>
      </div>
      <div class="book p-4">
        <h3>Book an Appointment</h3>
        <form action="#" method="POST" id="appointment-form" class="appointment-form">
        <input type="hidden" id="service_id" name="service_id" value="<?php echo
$service_id; ?>">
        <input type="hidden" id="user_id" name="user_id" value="<?php echo $user_id; ?>">
          <div class="d-md-flex">
            <div class="form-group">
```

```
            <input type="text" name="full_name" id="full_name" class="form-control"
placeholder="Full Name">
          </div>
        </div>
        <div class="d-md-flex">
          <div class="form-group">
            <div class="input-wrap">
              <!-- <div class="icon"><span class="ion-md-calendar"></span></div> -->
              <input type="date" name="appointment_date" id="appointment_date"
class="form-control"
                placeholder="Date">
            </div>
          </div>
          <div class="form-group ml-md-4">
            <div class="input-wrap">
              <!-- <div class="icon"><span class="ion-ios-clock"></span></div>
            <input type="text" name="appointment_time" class="form-control
appointment_time" placeholder="Time"> -->

              <select id="appointment_time" name="appointment_time" required>
                <option value="">Select Time</option>
                <option value="04:00 AM">04:00 AM</option>
                <option value="05:00 AM">05:00 AM</option>
                <option value="06:00 AM">06:00 AM</option>
                <option value="07:00 AM">07:00 AM</option>
                <option value="08:00 AM">08:00 AM</option>
                <option value="09:00 AM">09:00 AM</option>
                <option value="10:00 AM">10:00 AM</option>
                <option value="11:00 AM">11:00 AM</option>
                <option value="12:00 PM">12:00 PM</option>
                <option value="01:00 PM">01:00 PM</option>
                <option value="02:00 PM">02:00 PM</option>
                <option value="03:00 PM">03:00 PM</option>
```

```
                <option value="04:00 PM">04:00 PM</option>
                <option value="05:00 PM">05:00 PM</option>
                <option value="06:00 PM">06:00 PM</option>
                <option value="07:00 PM">07:00 PM</option>
                <option value="08:00 PM">08:00 PM</option>
                <option value="09:00 PM">09:00 PM</option>
              </select>


            </div>
          </div>
          <div class="form-group ml-md-4">
            <input type="text" name="phone" id="phone" class="form-control"
placeholder="Phone">
          </div>
        </div>
        <div class="d-md-flex">
          <div class="form-group">
            <textarea name="message" id="message" cols="30" rows="2" class="form-control"
placeholder="Message"></textarea>
          </div>
          <div class="form-group ml-md-4">
            <!-- <input type="submit" value="Appointment" class="btn btn-white py-3 px-4"> -
->
            <input type="hidden" id="appointment_amount" value="<?php echo
$row['price'];?>"> <!-- Example amount -->
      <button type="button" id="rzp-button">Pay & Book Appointment</button>
          </div>
        </div>
      </form>
    </div>
  </div>
 </section>
 <footer class="ftco-footer ftco-section img">
```

```
<div class="overlay"></div>
<div class="container">
 <div class="row mb-5">
  <div class="col-lg-3 col-md-6 mb-5 mb-md-5">
   <div class="ftco-footer-widget mb-4">
    <h2 class="ftco-heading-2">About Us</h2>
    <p>We offers a relaxing atmosphere with professional services, including haircuts, facials, and manicures.
     The salon is known for its friendly staff, quality treatments, and attention to detail.</p>
    <ul class="ftco-footer-social list-unstyled float-md-left float-lft mt-5">
     <li class="ftco-animate"><a href="#"><span class="icon-twitter"></span></a></li>
     <li class="ftco-animate"><a href="#"><span class="icon-facebook"></span></a></li>
     <li class="ftco-animate"><a href="#"><span class="icon-instagram"></span></a></li>
    </ul>
   </div>
  </div>
  <div class="col-lg-2 col-md-6 mb-5 mb-md-5">
   <div class="ftco-footer-widget mb-4 ml-md-4">
    <h2 class="ftco-heading-2">Services</h2>
    <ul class="list-unstyled">
     <li><a href="#" class="py-2 d-block">Haircut</a></li>
     <li><a href="#" class="py-2 d-block">Facial</a></li>
     <li><a href="#" class="py-2 d-block">Manicure</a></li>
     <li><a href="#" class="py-2 d-block">Makeup</a></li>
    </ul>
   </div>
  </div>
  <div class="col-lg-3 col-md-6 mb-5 mb-md-5">
   <div class="ftco-footer-widget mb-4">
    <h2 class="ftco-heading-2">Have a Questions?</h2>
    <div class="block-23 mb-3">
```

```
          <ul>
            <li><span class="icon icon-map-marker"></span><span class="text"> Hill top
               Street<br>Nilambur,<br>Malappuram,<br>Kerala,<br>India</span></li>
            <li><a href="#"><span class="icon icon-phone"></span><span class="text">+2
392 3929 210</span></a></li>
            <li><a href="#"><span class="icon icon-envelope"></span><span
                 class="text">info@yourdomain.com</span></a></li>
          </ul>
        </div>
      </div>
    </div>
  </div>
  <div class="row">
    <div class="col-md-12 text-center">
      <!-- loader -->
      <div id="ftco-loader" class="show fullscreen"><svg class="circular" width="48px"
height="48px">
          <circle class="path-bg" cx="24" cy="24" r="22" fill="none" stroke-width="4"
stroke="#eeeeee" />
          <circle class="path" cx="24" cy="24" r="22" fill="none" stroke-width="4" stroke-
miterlimit="10"
            stroke="#F96D00" />
        </svg></div>
      <!-- Sweet Alert -->
      <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script>
      <script src="js/jquery.min.js"></script>
      <script src="js/jquery-migrate-3.0.1.min.js"></script>
      <script src="js/popper.min.js"></script>
      <script src="js/bootstrap.min.js"></script>
      <script src="js/jquery.easing.1.3.js"></script>
      <script src="js/jquery.waypoints.min.js"></script>
      <script src="js/jquery.stellar.min.js"></script>
      <script src="js/owl.carousel.min.js"></script>
      <script src="js/jquery.magnific-popup.min.js"></script>
```

```
        <script src="js/aos.js"></script>
        <script src="js/jquery.animateNumber.min.js"></script>
        <script src="js/bootstrap-datepicker.js"></script>
        <script src="js/jquery.timepicker.min.js"></script>
        <script src="js/scrollax.min.js"></script>
        <script
          src="https://maps.googleapis.com/maps/api/js?key=AIzaSyBVWaKrjvy3MaE7SQ74_uJiULgl1JY0H2s&sensor=false"></script>
        <script src="js/google-map.js"></script>
        <script src="js/main.js"></script>
        <script src="https://checkout.razorpay.com/v1/checkout.js"></script>
        <!-- jQuery & Validation Plugin -->
        <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
        <script src="https://cdnjs.cloudflare.com/ajax/libs/jquery-validate/1.19.5/jquery.validate.min.js"></script>
        <script>
          $(document).ready(function () {
            // Custom validation methods
            jQuery.validator.addMethod('lettersonly', function (value, element) {
              return /^[^-\s][a-zA-Z_\s-]+$/.test(value);
            }, "Please use letters only.");
            jQuery.validator.addMethod('indianPhone', function (value, element) {
              return /^[6-9]\d{9}$/.test(value);
            }, "Please enter a valid Indian phone number starting with 6, 7, 8, or 9.");x`
            jQuery.validator.addMethod('validDate', function (value, element) {
    let selectedDate = new Date(value);
    let today = new Date();
    today.setHours(0, 0, 0, 0); // Reset time to midnight for comparison
    let oneYearFromToday = new Date();
    oneYearFromToday.setFullYear(today.getFullYear() + 1); // Set to exactly 1 year from today
    return selectedDate >= today && selectedDate <= oneYearFromToday;
}, "Please seleuct a date between today and 1 year from now.");
```

```
jQuery.validator.addMethod('validTime', function (value, element) {
  let selectedTime = value.split(':');
  let hours = parseInt(selectedTime[0], 10);
  let minutes = parseInt(selectedTime[1], 10);
  return (hours >= 4 && hours < 22) || (hours === 22 && minutes === 0);
}, "Please select a time between 4 AM and 10 PM.");
$(".appointment-form").validate({
  rules: {
    full_name: {
      lettersonly: true,
      required: true,
      minlength: 3
    },
    appointment_date: {
      required: true,
      date: true,
      validDate: true
    },
    appointment_time: {
      required: true,
      validTime: true
    },
    phone: {
      required: true,
      digits: true,
      minlength: 10,
      maxlength: 10,
      indianPhone: true
    },
    message: {
      minlength: 5
    }
```

```
  },
  messages: {
   full_name: {
    required: "Please enter your full name",
    lettersonly: "Name must be in alphabets only",
    minlength: "Name should be at least 3 characters long"
   },
   appointment_date: {
    required: "Please select a date",
    validDate: "Please select a date between today and 1 year from now"
   },
   appointment_time: {
    required: "Please select a time",
    validTime: "Time should be between 4 AM and 10 PM"
   },
   phone: {
    required: "Phone number is required",
    digits: "Please enter only numbers",
    minlength: "Invalid phone number",
    maxlength: "Invalid phone number",
    indianPhone: "Please enter a valid phone number starting with 6, 7, 8, or 9."
   },
   message: {
    minlength: "Message should be at least 5 characters long"
   }
  },
  errorElement: "div",
  errorPlacement: function (error, element) {
   error.addClass("text-danger");
   error.insertAfter(element);
  }
 });
```

```
// Prevent selecting past dates in the input field
let today = new Date().toISOString().split("T")[0];
$("#appointment_date").attr("min", today);

// Handle date change to fetch booked time slots
$("#appointment_date").on('change', function() {
  var selectedDate = $(this).val();
  if(selectedDate) {
    // Make AJAX call to get booked times
    $.ajax({
      type: "POST",
      url: "get_booked_times.php",
      data: { date: selectedDate },
      dataType: "json",
      success: function(response) {
        // Enable all time slots first
        $("#appointment_time option").prop("disabled", false);

        // Disable booked time slots
        if(response.booked_times) {
          $.each(response.booked_times, function(index, time) {
            $("#appointment_time option[value='" + time + "']").prop("disabled", true);
          });
        }
                        // If the currently selected time is now disabled, reset selection
        if($("#appointment_time").val() && $("#appointment_time
option:selected").prop("disabled")) {
          $("#appointment_time").val("");
        }
      },
      error: function(xhr, status, error) {
        console.error("Error fetching booked times:", error);
```

```
            }
          });
        }
      });
    });

    document.getElementById('rzp-button').onclick = function (e) {
        e.preventDefault();
        var form = document.getElementById('appointment-form');
        if (!form.checkValidity()) {
            alert('Please fill all required fields');
            return;
        }
        var userId = document.getElementById('user_id').value;
        var serviceId = document.getElementById('service_id').value;
        var fullName = document.getElementById('full_name').value;
        var appointmentDate = document.getElementById('appointment_date').value;
        var appointmentTime = document.getElementById('appointment_time').value;
        var phone = document.getElementById('phone').value;
        var message = document.getElementById('message').value;
        var eamount = document.getElementById('appointment_amount').value * 100; // Convert INR to paise
        var amount=eamount/2;
        // Check if the selected time slot is still available
        $.ajax({
            type: "POST",
            url: "check_time_availability.php",
            data: {
                date: appointmentDate,
                time: appointmentTime
            },
            dataType: "json",
            success: function(response) {
```

```
                if(response.available) {
                    // Proceed with payment if time slot is available
                    initiatePayment(userId, serviceId, fullName, appointmentDate,
appointmentTime, phone, message, amount);
                } else {
                    Swal.fire({
                        title: "Time Slot Unavailable",
                        text: "Sorry, this time slot has just been booked by someone else. Please
choose another time.",
                        icon: "warning",
                        confirmButtonText: "OK"
                    });
                                        // Refresh available time slots
                    $("#appointment_date").trigger('change');
                }
            },
            error: function(xhr, status, error) {
                console.error("Error checking time availability:", error);
                Swal.fire({
                    title: "Error",
                    text: "Could not verify time slot availability. Please try again.",
                    icon: "error",
                    confirmButtonText: "OK"
                });
            }
        });
    };

    function initiatePayment(userId, serviceId, fullName, appointmentDate,
appointmentTime, phone, message, amount) {
        var options = {
            "key": "rzp_test_s8u2UQ54kE7TBA",
            "amount": amount,
```

```
"currency": "INR",
"name": "Serenity Styles",
"description": "Appointment Booking",
"handler": function (response) {
  $.ajax({
    type: "POST",
    url: "booknow_process.php",
    data: {
      user_id: userId,
      service_id: serviceId,
      name: fullName,
      appointment_date: appointmentDate,
      appointment_time: appointmentTime,
      phone: phone,
      message: message,
      amount: amount / 100, // Convert back to INR
      payment_id: response.razorpay_payment_id,
      status: 'Pending'
    },
    success: function (result) {
      console.log(result);
      if (result.trim() === "success") {
        Swal.fire({
          title: "Payment Successful!",
          text: "Wait for Appointment Confirmation.",
          icon: "success",
          confirmButtonText: "OK"
        }).then(() => {
          window.location.href = "booknow.php";
        });
      } else {
        Swal.fire({
```

```
                        title: "Error!",
                        text: "Server error.Try again!",
                        icon: "error",
                        confirmButtonText: "OK"
                    });
                }
            }
        });
    },
    "prefill": {
        "name": fullName,
        "contact": phone
    },
    "theme": {
        "color": "#c49b63"
    }
};

var rzp1 = new Razorpay(options);
rzp1.open();
rzp1.on('payment.failed', function (response) {
    alert("Payment Failed: " + response.error.description);
});
}
</script>

<?php
// Display the alert after the page loads
echo $alertMessage;
?>
</body>
</html>
```

Add to Cart

```
<?php
include('connect.php');
if(isset($_POST['product_id'])){
   $product_id = $_POST['product_id'];
   $user_id = $_SESSION['user_id'];
   // Check if the product already exists in the cart
   $sql = "SELECT * FROM tbl_cart WHERE user_id = $user_id AND product_id = $product_id AND status='Active'";
   $result = $conn->query($sql);
   if ($result->num_rows > 0) {
      // Product exists in the cart, update the quantity
      $row = $result->fetch_assoc();
      $new_quantity = $row['quantity'] + 1;
      $sql = "UPDATE tbl_cart SET quantity = $new_quantity WHERE user_id = $user_id AND product_id = $product_id";
   } else {
      // Product does not exist in the cart, insert a new row
      $sql = "SELECT price FROM tbl_products WHERE product_id = $product_id";
      $result = $conn->query($sql);
      $row = $result->fetch_assoc();
      $price=$row["price"];


 $sql = "INSERT INTO tbl_cart (user_id, product_id, quantity,unit_price) VALUES ($user_id, $product_id, 1,$price)";
   }
   if ($conn->query($sql)) {
            echo "<script>
           setTimeout(function() {
              Swal.fire({
                 icon: 'success',
                 title: 'Added to Cart!',
                 text: 'Product added to cart successfully!',
```

```
                confirmButtonText: 'OK'
            }).then(() => {
                window.location.href = 'http://localhost/coffeeduplicate/shop.php';
            });
        }, 500);
    </script>";
  }
}
?>
<!-- Sweet Alert -->
<script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script
```

9.2 SCREEN SHOTS

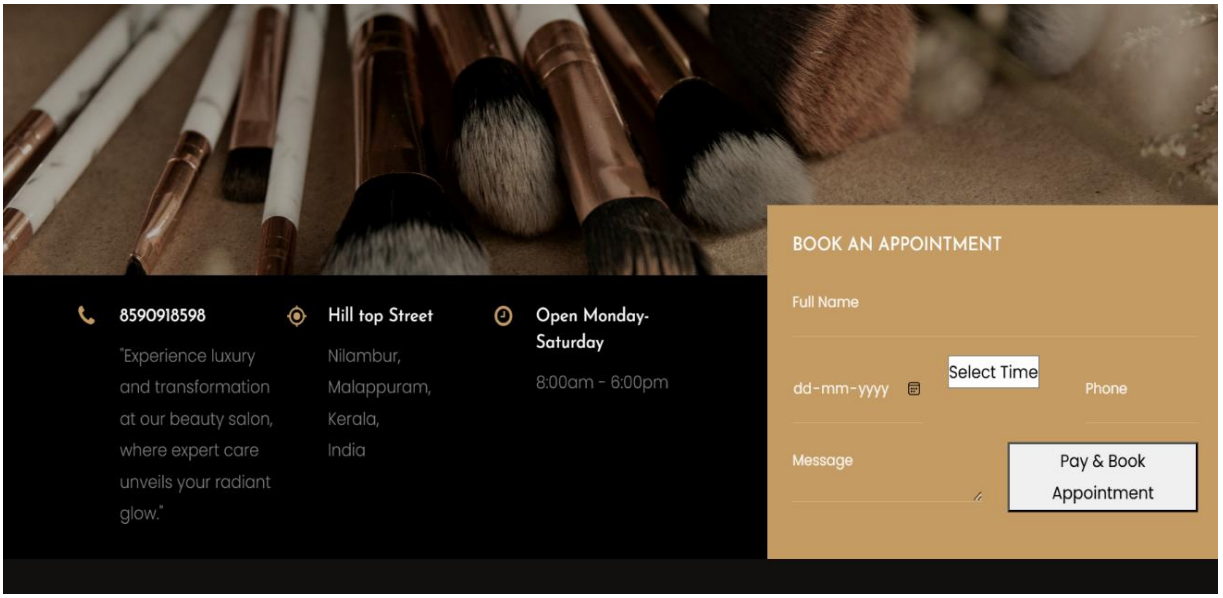


Figure 22 : Book Appointment Page

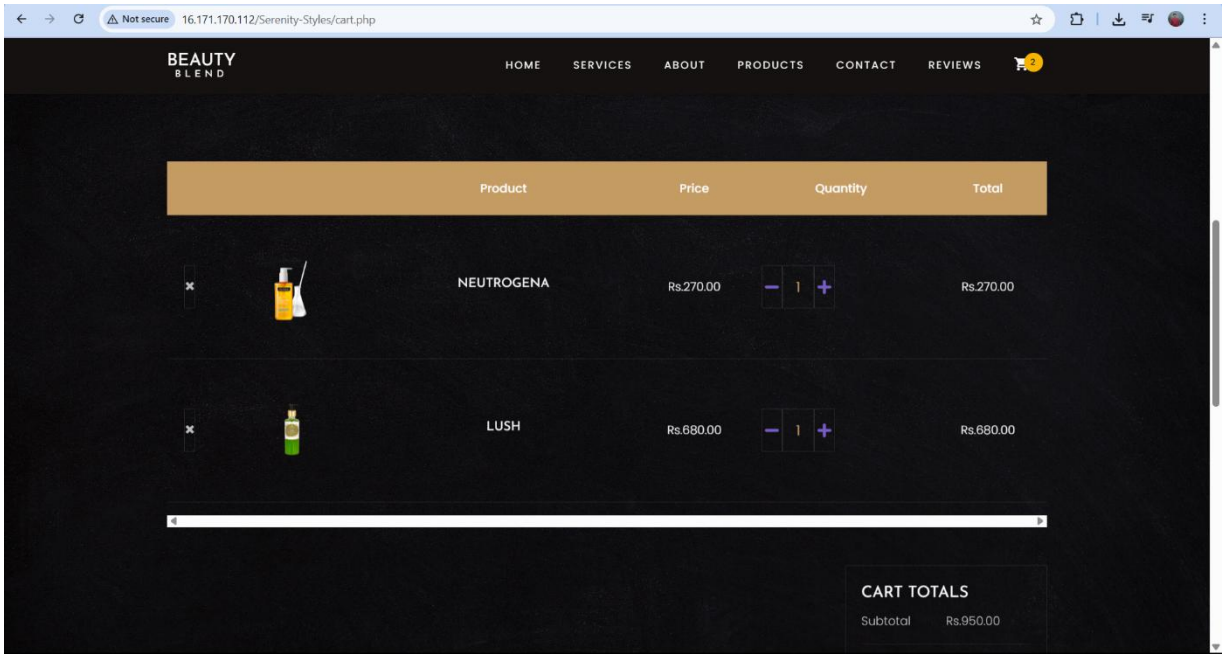


Figure 23 : Cart Page

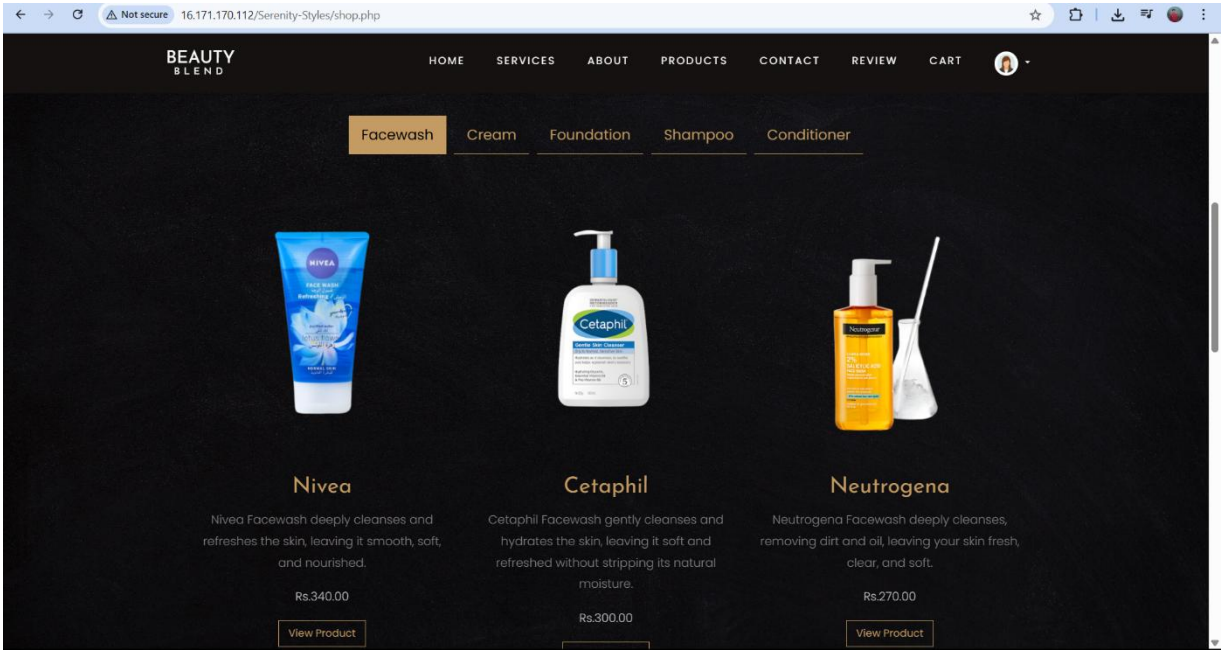


Figure 24 : Product Page

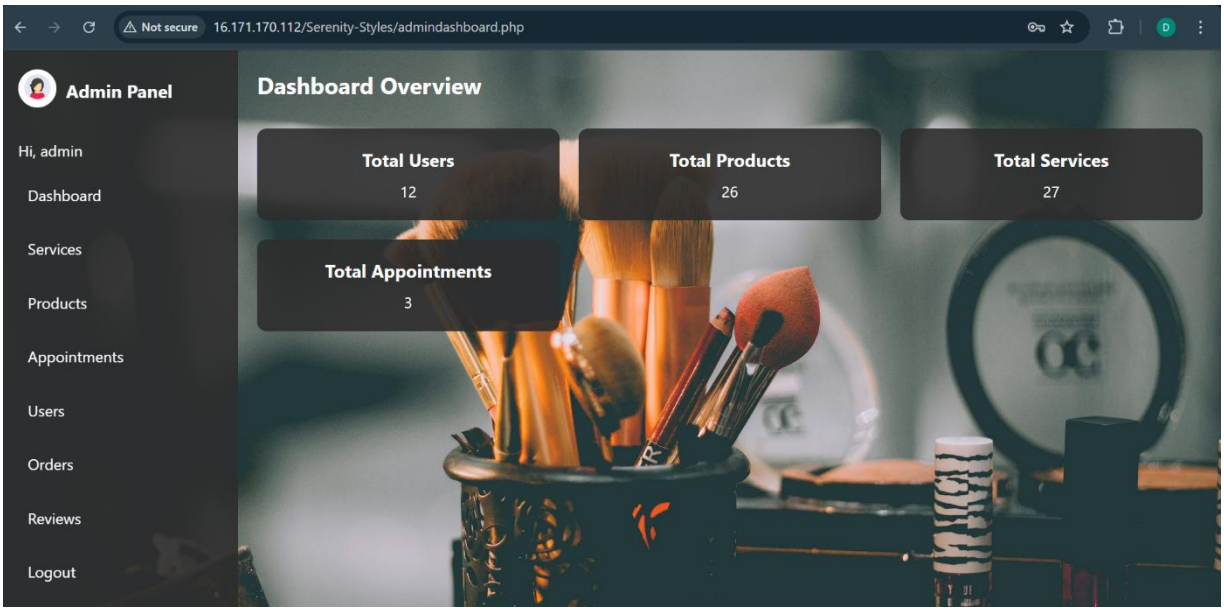


Figure 25 : Add Product Page

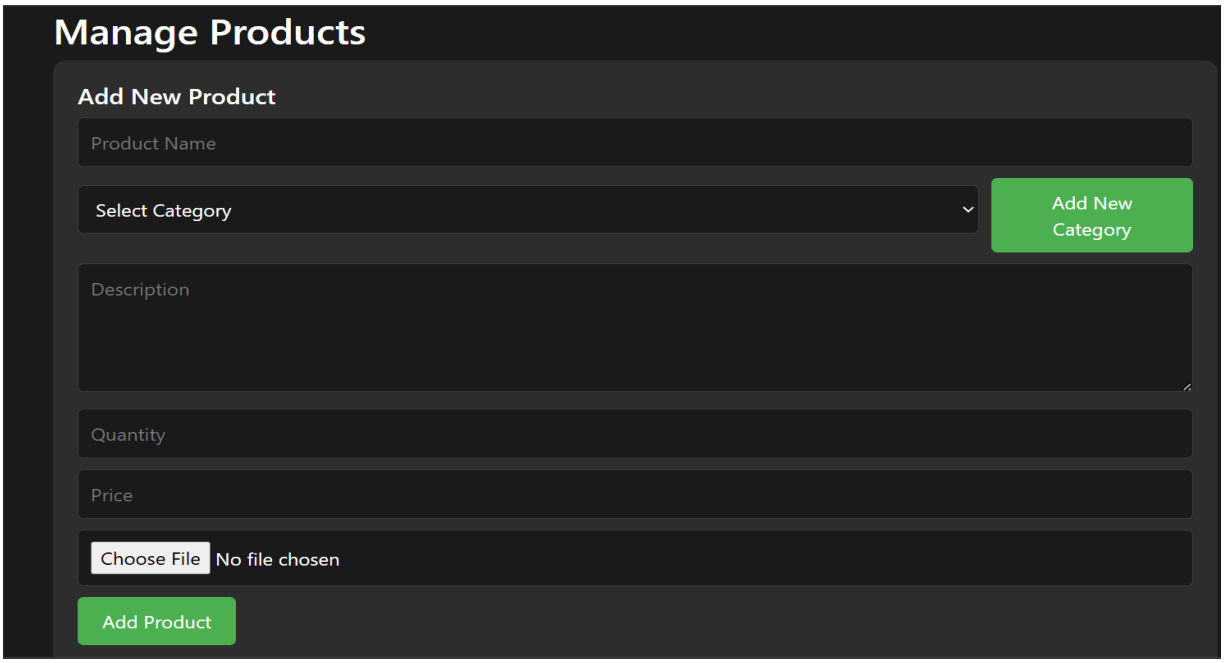


Figure 26 : Add Product Page